

Chapter 1

Introduction

Programming languages are notations for describing computations to people and to machines. The world as we know it depends on programming languages because all the software running on all the computers was written in some programming language. But before a program can be run, it first must be translated into a form in which it can be executed by a computer.

The software systems that do this translation are called *compilers*.

This book is about how to design and implement compilers. We shall discover that a few basic ideas can be used to construct translators for a wide variety of languages and machines. Besides compilers, the principles and techniques for compiler design are applicable to so many other domains that they are likely to be reused many times in the career of a computer scientist. The study of compiler writing touches upon programming languages, machine architecture, language theory, algorithms, and software engineering.

In this preliminary chapter, we introduce the different forms of language translators, give a high level overview of the structure of a typical compiler, and discuss the trends in programming languages and machine architecture that are shaping compilers. We include some observations on the relationship between compiler design and computer science theory and an outline of the applications of compiler technology that go beyond compilation. We end with a brief outline of key programming language concepts that will be needed for our study of compilers.

1.1 Language Processors

Simply stated, a compiler is a program that can read a program in one language—the *source* language—and translate it into an equivalent program in another language—the *target* language (see Fig. 1.1). An important role of the compiler is to report any errors in the source program that it detects during the translation process.

CHAPTER 1 INTRODUCTION

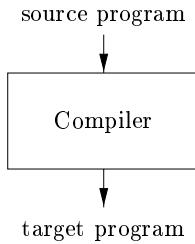


Figure 1.1 A compiler

If the target program is an executable machine language program, it can then be called by the user to process inputs and produce outputs (see Fig. 1.2).

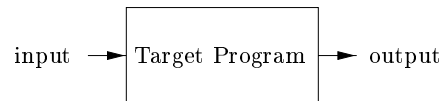


Figure 1.2 Running the target program

An *interpreter* is another common kind of language processor. Instead of producing a target program as a translation, an interpreter appears to directly execute the operations specified in the source program on inputs supplied by the user, as shown in Fig. 1.3.

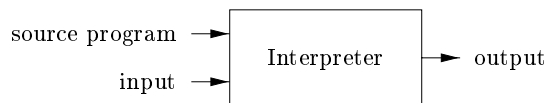


Figure 1.3 An interpreter

The machine language target program produced by a compiler is usually much faster than an interpreter at mapping inputs to outputs. An interpreter, however, can usually give better error diagnostics than a compiler, because it executes the source program statement by statement.

Example 1.1 Java language processors combine compilation and interpretation, as shown in Fig. 1.4. A Java source program may first be compiled into an intermediate form called *bytecodes*. The bytecodes are then interpreted by a virtual machine. A benefit of this arrangement is that bytecodes compiled on one machine can be interpreted on another machine, perhaps across a network.

In order to achieve faster processing of inputs to outputs, some Java compilers, called *just in time* compilers, translate the bytecodes into machine language immediately before they run the intermediate program to process the input. □

1.1 LANGUAGE PROCESSORS

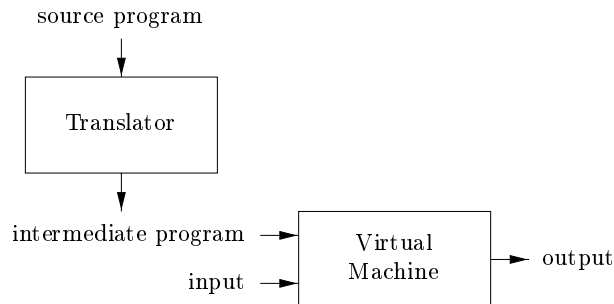


Figure 1.4 A hybrid compiler

In addition to a compiler, several other programs may be required to create an executable target program, as shown in Fig. 1.5. A source program may be divided into modules stored in separate files. The task of collecting the source program is sometimes entrusted to a separate program called a *preprocessor*. The preprocessor may also expand shorthands, called macros, into source language statements.

The modified source program is then fed to a compiler. The compiler may produce an assembly language program as its output, because assembly language is easier to produce as output and is easier to debug. The assembly language is then processed by a program called an *assembler* that produces relocatable machine code as its output.

Large programs are often compiled in pieces, so the relocatable machine code may have to be linked together with other relocatable object files and library files into the code that actually runs on the machine. The *linker* resolves external memory addresses, where the code in one file may refer to a location in another file. The *loader* then puts together all of the executable object files into memory for execution.

1.1.1 Exercises for Section 1.1

Exercise 1.1.1 What is the difference between a compiler and an interpreter?

Exercise 1.1.2 What are the advantages of
a) a compiler over an interpreter
b) an interpreter over a compiler?

Exercise 1.1.3 What advantages are there to a language processing system in which the compiler produces assembly language rather than machine language?

Exercise 1.1.4 A compiler that translates a high level language into another high level language is called a *source to source* translator. What advantages are there to using C as a target language for a compiler?

Exercise 1.1.5 Describe some of the tasks that an assembler needs to perform.

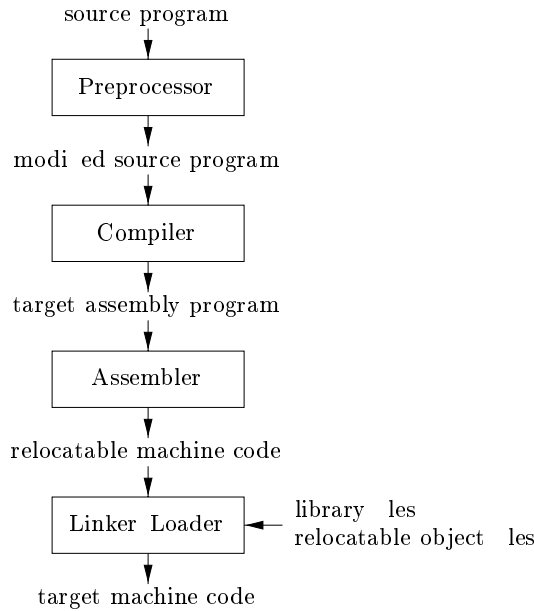


Figure 1.5 A language processing system

1.2 The Structure of a Compiler

Up to this point we have treated a compiler as a single box that maps a source program into a semantically equivalent target program. If we open up this box a little, we see that there are two parts to this mapping—analysis and synthesis.

The *analysis* part breaks up the source program into constituent pieces and imposes a grammatical structure on them. It then uses this structure to create an intermediate representation of the source program. If the analysis part detects that the source program is either syntactically ill-formed or semantically unsound, then it must provide informative messages so the user can take corrective action. The analysis part also collects information about the source program and stores it in a data structure called a *symbol table*, which is passed along with the intermediate representation to the synthesis part.

The *synthesis* part constructs the desired target program from the intermediate representation and the information in the symbol table. The analysis part is often called the *front end* of the compiler; the synthesis part is the *back end*.

If we examine the compilation process in more detail, we see that it operates as a sequence of *phases*, each of which transforms one representation of the source program to another. A typical decomposition of a compiler into phases is shown in Fig. 1.6. In practice, several phases may be grouped together, and the intermediate representations between the grouped phases need not be constructed explicitly. The symbol table, which stores information about the

1.2 THE STRUCTURE OF A COMPILER

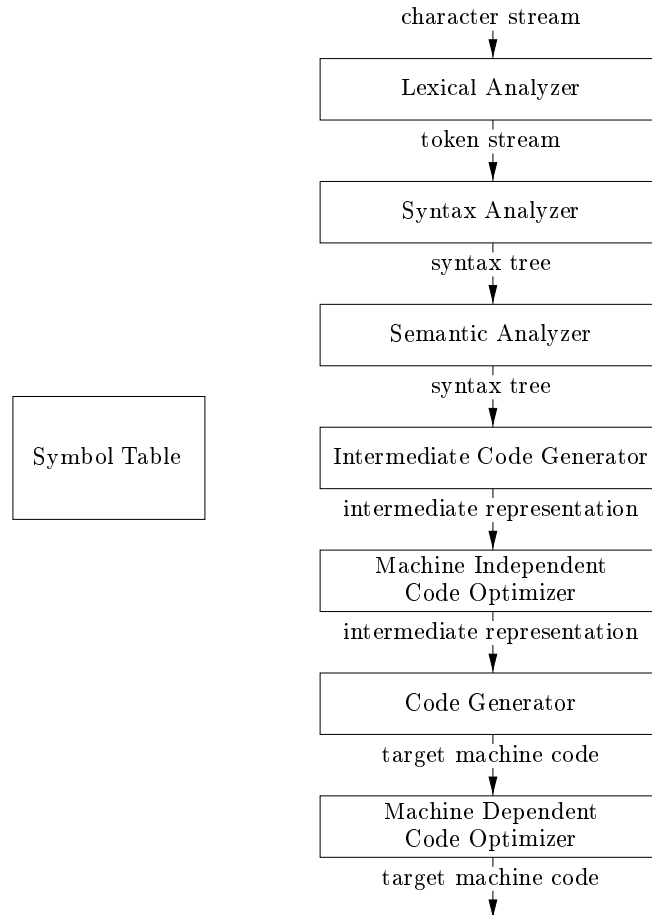


Figure 1.6 Phases of a compiler

entire source program is used by all phases of the compiler

Some compilers have a machine independent optimization phase between the front end and the back end. The purpose of this optimization phase is to perform transformations on the intermediate representation so that the back end can produce a better target program than it would have otherwise produced from an unoptimized intermediate representation. Since optimization is optional, one or the other of the two optimization phases shown in Fig. 1.6 may be missing.

1.2.1 Lexical Analysis

The first phase of a compiler is called *lexical analysis* or *scanning*. The lexical analyzer reads the stream of characters making up the source program

CHAPTER 1 INTRODUCTION

and groups the characters into meaningful sequences called *lexemes*. For each lexeme the lexical analyzer produces as output a *token* of the form

$\langle \textit{token name attribute value} \rangle$

that it passes on to the subsequent phase: syntax analysis. In the token the first component *token name* is an abstract symbol that is used during syntax analysis, and the second component *attribute value* points to an entry in the symbol table for this token. Information from the symbol table entry is needed for semantic analysis and code generation.

For example, suppose a source program contains the assignment statement

position initial rate 60 1 1

The characters in this assignment could be grouped into the following lexemes and mapped into the following tokens passed on to the syntax analyzer:

- 1 **position** is a lexeme that would be mapped into a token $\langle \mathbf{id} \ 1 \rangle$ where **id** is an abstract symbol standing for *identifier* and 1 points to the symbol table entry for **position**. The symbol table entry for an identifier holds information about the identifier, such as its name and type.
- 2 The assignment symbol **=** is a lexeme that is mapped into the token $\langle \ \rangle$. Since this token needs no attribute value, we have omitted the second component. We could have used any abstract symbol such as **assign** for the token name, but for notational convenience we have chosen to use the lexeme itself as the name of the abstract symbol.
- 3 **initial** is a lexeme that is mapped into the token $\langle \mathbf{id} \ 2 \rangle$ where 2 points to the symbol table entry for **initial**.
- 4 **+** is a lexeme that is mapped into the token $\langle \ + \ \rangle$.
- 5 **rate** is a lexeme that is mapped into the token $\langle \mathbf{id} \ 3 \rangle$ where 3 points to the symbol table entry for **rate**.
- 6 ***** is a lexeme that is mapped into the token $\langle \ * \ \rangle$.
- 7 **60** is a lexeme that is mapped into the token $\langle 60 \rangle$.¹

Blanks separating the lexemes would be discarded by the lexical analyzer.

Figure 1.7 shows the representation of the assignment statement `1 1` after lexical analysis as the sequence of tokens:

$\langle \mathbf{id} \ 1 \rangle \ \langle \ \rangle \langle \mathbf{id} \ 2 \rangle \ \langle \ + \ \rangle \langle \mathbf{id} \ 3 \rangle \ \langle \ * \ \rangle \langle 60 \rangle$ 1 2

In this representation, the token names `id` and `+` are abstract symbols for the assignment, addition, and multiplication operators, respectively.

¹Technically speaking, for the lexeme **60** we should make up a token like $\langle \mathbf{number} \ 4 \rangle$ where 4 points to the symbol table for the internal representation of integer 60, but we shall defer the discussion of tokens for numbers until Chapter 2. Chapter 3 discusses techniques for building lexical analyzers.

12 THE STRUCTURE OF A COMPILER

1	position	
2	initial	
3	rate	

SYMBOL TABLE

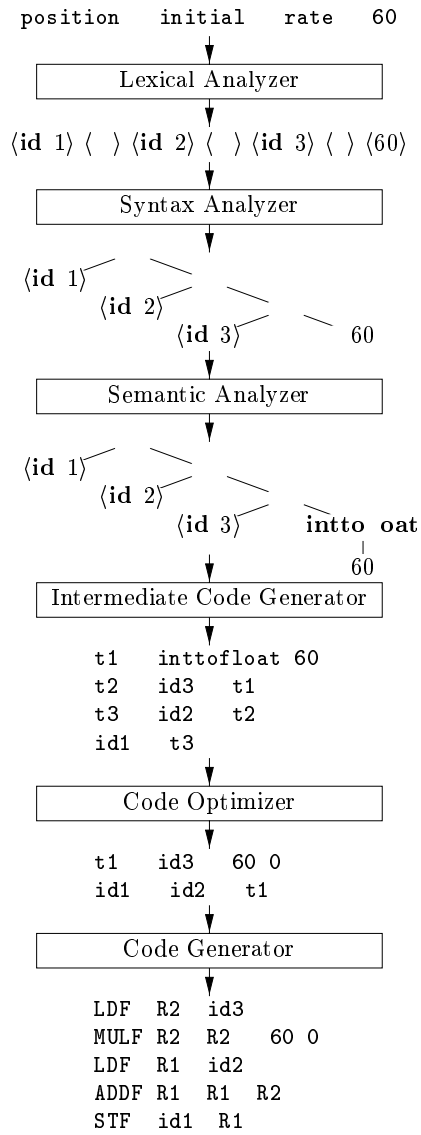


Figure 1.7 Translation of an assignment statement

1.2.2 Syntax Analysis

The second phase of the compiler is *syntax analysis* or *parsing*. The parser uses the first components of the tokens produced by the lexical analyzer to create a tree-like intermediate representation that depicts the grammatical structure of the token stream. A typical representation is a *syntax tree* in which each interior node represents an operation and the children of the node represent the arguments of the operation. A syntax tree for the token stream 1.2 is shown as the output of the syntactic analyzer in Fig. 1.7.

This tree shows the order in which the operations in the assignment

```
position = initial + rate * 60
```

are to be performed. The tree has an interior node labeled `+` with `<id 3>` as its left child and the integer 60 as its right child. The node `<id 3>` represents the identifier `rate`. The node labeled `*` makes it explicit that we must first multiply the value of `rate` by 60. The node labeled `+` indicates that we must add the result of this multiplication to the value of `initial`. The root of the tree, labeled `=`, indicates that we must store the result of this addition into the location for the identifier `position`. This ordering of operations is consistent with the usual conventions of arithmetic which tell us that multiplication has higher precedence than addition, and hence that the multiplication is to be performed before the addition.

The subsequent phases of the compiler use the grammatical structure to help analyze the source program and generate the target program. In Chapter 4 we shall use context-free grammars to specify the grammatical structure of programming languages and discuss algorithms for constructing efficient syntax analyzers automatically from certain classes of grammars. In Chapters 2 and 5 we shall see that syntax-directed definitions can help specify the translation of programming language constructs.

1.2.3 Semantic Analysis

The *semantic analyzer* uses the syntax tree and the information in the symbol table to check the source program for semantic consistency with the language definition. It also gathers type information and saves it in either the syntax tree or the symbol table for subsequent use during intermediate code generation.

An important part of semantic analysis is *type checking*, where the compiler checks that each operator has matching operands. For example, many programming language definitions require an array index to be an integer; the compiler must report an error if a floating-point number is used to index an array.

The language specification may permit some type conversions called *coercions*. For example, a binary arithmetic operator may be applied to either a pair of integers or to a pair of floating-point numbers. If the operator is applied to a floating-point number and an integer, the compiler may convert or coerce the integer into a floating-point number.

1.2 THE STRUCTURE OF A COMPILER

Such a coercion appears in Fig. 1.7. Suppose that `position`, `initial`, and `rate` have been declared to be floating point numbers, and that the lexeme `60` by itself forms an integer. The type checker in the semantic analyzer in Fig. 1.7 discovers that the operator `+` is applied to a floating point number `rate` and an integer `60`. In this case, the integer may be converted into a floating point number. In Fig. 1.7, notice that the output of the semantic analyzer has an extra node for the operator `intto float`, which explicitly converts its integer argument into a floating point number. Type checking and semantic analysis are discussed in Chapter 6.

1.2.4 Intermediate Code Generation

In the process of translating a source program into target code, a compiler may construct one or more intermediate representations, which can have a variety of forms. Syntax trees are a form of intermediate representation; they are commonly used during syntax and semantic analysis.

After syntax and semantic analysis of the source program, many compilers generate an explicit low-level or machine-like intermediate representation, which we can think of as a program for an abstract machine. This intermediate representation should have two important properties: it should be easy to produce, and it should be easy to translate into the target machine.

In Chapter 6, we consider an intermediate form called *three address code*, which consists of a sequence of assembly-like instructions with three operands per instruction. Each operand can act like a register. The output of the intermediate code generator in Fig. 1.7 consists of the three address code sequence

```
t1  inttofloat 60
t2  id3   t1
t3  id2   t2
id1  t3
```

1.3

There are several points worth noting about three address instructions. First, each three address assignment instruction has at most one operator on the right side. Thus, these instructions fix the order in which operations are to be done: the multiplication precedes the addition in the source program 1.1. Second, the compiler must generate a temporary name to hold the value computed by a three address instruction. Third, some three address instructions, like the first and last in the sequence 1.3 above, have fewer than three operands.

In Chapter 6, we cover the principal intermediate representations used in compilers. Chapter 5 introduces techniques for syntax-directed translation that are applied in Chapter 6 to type checking and intermediate code generation for typical programming language constructs such as expressions, flow of control constructs, and procedure calls.

1.2.5 Code Optimization

The machine independent code optimization phase attempts to improve the intermediate code so that better target code will result. Usually better means faster, but other objectives may be desired, such as shorter code, or target code that consumes less power. For example, a straightforward algorithm generates the intermediate code 1.3 using an instruction for each operator in the tree representation that comes from the semantic analyzer.

A simple intermediate code generation algorithm followed by code optimization is a reasonable way to generate good target code. The optimizer can deduce that the conversion of 60 from integer to floating point can be done once and for all at compile time, so the **into oat** operation can be eliminated by replacing the integer 60 by the floating point number 60.0. Moreover, **t3** is used only once to transmit its value to **id1**, so the optimizer can transform 1.3 into the shorter sequence

```

t1  id3  60.0
id1  id2  t1
1.4
```

There is a great variation in the amount of code optimization different compilers perform. In those that do the most, the so-called optimizing compilers, a significant amount of time is spent on this phase. There are simple optimizations that significantly improve the running time of the target program without slowing down compilation too much. The chapters from 8 on discuss machine independent and machine dependent optimizations in detail.

1.2.6 Code Generation

The code generator takes as input an intermediate representation of the source program and maps it into the target language. If the target language is machine code, registers or memory locations are selected for each of the variables used by the program. Then, the intermediate instructions are translated into sequences of machine instructions that perform the same task. A crucial aspect of code generation is the judicious assignment of registers to hold variables.

For example, using registers **R1** and **R2**, the intermediate code in 1.4 might get translated into the machine code

```

LDF  R2  id3
MULF R2  R2  60.0
LDF  R1  id2
ADDF R1  R1  R2
STF  id1  R1
1.5
```

The first operand of each instruction specifies a destination. The **F** in each instruction tells us that it deals with floating point numbers. The code in

1.2 THE STRUCTURE OF A COMPILER

1.5 loads the contents of address `id3` into register `R2` then multiplies it with floating point constant `60.0`. The `signi` signifies that `60.0` is to be treated as an immediate constant. The third instruction moves `id2` into register `R1` and the fourth adds to it the value previously computed in register `R2`. Finally the value in register `R1` is stored into the address of `id1` so the code correctly implements the assignment statement 1.1. Chapter 8 covers code generation.

This discussion of code generation has ignored the important issue of storage allocation for the identifiers in the source program. As we shall see in Chapter 7, the organization of storage at run time depends on the language being compiled. Storage allocation decisions are made either during intermediate code generation or during code generation.

1.2.7 Symbol Table Management

An essential function of a compiler is to record the variable names used in the source program and collect information about various attributes of each name. These attributes may provide information about the storage allocated for a name, its type, its scope, where in the program its value may be used, and in the case of procedure names, such things as the number and types of its arguments, the method of passing each argument, for example, by value or by reference, and the type returned.

The symbol table is a data structure containing a record for each variable name, with fields for the attributes of the name. The data structure should be designed to allow the compiler to find the record for each name quickly and to store or retrieve data from that record quickly. Symbol tables are discussed in Chapter 2.

1.2.8 The Grouping of Phases into Passes

The discussion of phases deals with the logical organization of a compiler. In an implementation, activities from several phases may be grouped together into a *pass* that reads an input file and writes an output file. For example, the front end phases of lexical analysis, syntax analysis, semantic analysis, and intermediate code generation might be grouped together into one pass. Code optimization might be an optional pass. Then there could be a back end pass consisting of code generation for a particular target machine.

Some compiler collections have been created around carefully designed intermediate representations that allow the front end for a particular language to interface with the back end for a certain target machine. With these collections we can produce compilers for different source languages for one target machine by combining different front ends with the back end for that target machine. Similarly, we can produce compilers for different target machines by combining a front end with back ends for different target machines.

1.2.9 Compiler Construction Tools

The compiler writer like any software developer can profitably use modern software development environments containing tools such as language editors, debuggers, version managers, profilers, test harnesses, and so on. In addition to these general software development tools, other more specialized tools have been created to help implement various phases of a compiler.

These tools use specialized languages for specifying and implementing specific components, and many use quite sophisticated algorithms. The most successful tools are those that hide the details of the generation algorithm and produce components that can be easily integrated into the remainder of the compiler. Some commonly used compiler construction tools include

- 1 *Parser generators* that automatically produce syntax analyzers from a grammatical description of a programming language
- 2 *Scanner generators* that produce lexical analyzers from a regular expression description of the tokens of a language
- 3 *Syntax directed translation engines* that produce collections of routines for walking a parse tree and generating intermediate code
- 4 *Code generator generators* that produce a code generator from a collection of rules for translating each operation of the intermediate language into the machine language for a target machine
- 5 *Data flow analysis engines* that facilitate the gathering of information about how values are transmitted from one part of a program to each other part. Data flow analysis is a key part of code optimization.
- 6 *Compiler construction toolkits* that provide an integrated set of routines for constructing various phases of a compiler.

We shall describe many of these tools throughout this book.

1.3 The Evolution of Programming Languages

The first electronic computers appeared in the 1940s and were programmed in machine language by sequences of 0s and 1s that explicitly told the computer what operations to execute and in what order. The operations themselves were very low level: move data from one location to another, add the contents of two registers, compare two values, and so on. Needless to say, this kind of programming was slow, tedious, and error prone. And once written, the programs were hard to understand and modify.

1.3 THE EVOLUTION OF PROGRAMMING LANGUAGES

1.3.1 The Move to Higher level Languages

The first step towards more people friendly programming languages was the development of mnemonic assembly languages in the early 1950s. Initially the instructions in an assembly language were just mnemonic representations of machine instructions. Later macro instructions were added to assembly languages so that a programmer could define parameterized shorthands for frequently used sequences of machine instructions.

A major step towards higher level languages was made in the latter half of the 1950s with the development of Fortran for scientific computation, Cobol for business data processing, and Lisp for symbolic computation. The philosophy behind these languages was to create higher level notations with which programmers could more easily write numerical computations, business applications, and symbolic programs. These languages were so successful that they are still in use today.

In the following decades many more languages were created with innovative features to help make programming easier, more natural, and more robust. Later in this chapter we shall discuss some key features that are common to many modern programming languages.

Today there are thousands of programming languages. They can be classified in a variety of ways. One classification is by generation. *First generation languages* are the machine languages, *second generation* the assembly languages, and *third generation* the higher level languages like Fortran, Cobol, Lisp, C, C++, and Java. *Fourth generation languages* are languages designed for specific applications like NOMAD for report generation, SQL for database queries, and Postscript for text formatting. The term *fifth generation language* has been applied to logic and constraint based languages like Prolog and OPS5.

Another classification of languages uses the term *imperative* for languages in which a program specifies *how* a computation is to be done and *declarative* for languages in which a program specifies *what* computation is to be done. Languages such as C, C++, and Java are imperative languages. In imperative languages there is a notion of program state and statements that change the state. Functional languages such as ML and Haskell and constraint logic languages such as Prolog are often considered to be declarative languages.

The term *von Neumann language* is applied to programming languages whose computational model is based on the von Neumann computer architecture. Many of today's languages, such as Fortran and C, are von Neumann languages.

An *object oriented language* is one that supports object oriented programming, a programming style in which a program consists of a collection of objects that interact with one another. Simula 67 and Smalltalk are the earliest major object oriented languages. Languages such as C++, Java, and Ruby are more recent object oriented languages.

Scripting languages are interpreted languages with high level operators designed for 'gluing together' computations. These computations were originally

called scripts. Awk, JavaScript, Perl, PHP, Python, Ruby, and Tcl are popular examples of scripting languages. Programs written in scripting languages are often much shorter than equivalent programs written in languages like C.

1.3.2 Impacts on Compilers

Since the design of programming languages and compilers are intimately related, the advances in programming languages placed new demands on compiler writers. They had to devise algorithms and representations to translate and support the new language features. Since the 1940s, computer architecture has evolved as well. Not only did the compiler writers have to track new language features, they also had to devise translation algorithms that would take maximal advantage of the new hardware capabilities.

Compilers can help promote the use of high-level languages by minimizing the execution overhead of the programs written in these languages. Compilers are also critical in making high-performance computer architectures effective on users' applications. In fact, the performance of a computer system is so dependent on compiler technology that compilers are used as a tool in evaluating architectural concepts before a computer is built.

Compiler writing is challenging. A compiler by itself is a large program. Moreover, many modern language processing systems handle several source languages and target machines within the same framework; that is, they serve as collections of compilers, possibly consisting of millions of lines of code. Consequently, good software engineering techniques are essential for creating and evolving modern language processors.

A compiler must translate correctly the potentially infinite set of programs that could be written in the source language. The problem of generating the optimal target code from a source program is undecidable in general; thus, compiler writers must evaluate tradeoffs about what problems to tackle and what heuristics to use to approach the problem of generating efficient code.

A study of compilers is also a study of how theory meets practice, as we shall see in Section 1.4.

The purpose of this text is to teach the methodology and fundamental ideas used in compiler design. It is not the intention of this text to teach all the algorithms and techniques that could be used for building a state-of-the-art language processing system. However, readers of this text will acquire the basic knowledge and understanding to learn how to build a compiler relatively easily.

1.3.3 Exercises for Section 1.3

Exercise 1.3.1 Indicate which of the following terms

- | | | |
|----------------------|----------------|---------------------|
| a. imperative | b. declarative | c. von Neumann |
| d. object oriented | e. functional | f. third generation |
| g. fourth generation | h. scripting | |

1.4 THE SCIENCE OF BUILDING A COMPILER

apply to which of the following languages

1 C	2 C	3 Cobol	4 Fortran	5 Java
6 Lisp	7 ML	8 Perl	9 Python	10 VB

1.4 The Science of Building a Compiler

Compiler design is full of beautiful examples where complicated real world problems are solved by abstracting the essence of the problem mathematically. These serve as excellent illustrations of how abstractions can be used to solve problems. take a problem, formulate a mathematical abstraction that captures the key characteristics, and solve it using mathematical techniques. The problem formulation must be grounded in a solid understanding of the characteristics of computer programs, and the solution must be validated and refined empirically.

A compiler must accept all source programs that conform to the specification of the language: the set of source programs is infinite and any program can be very large, consisting of possibly millions of lines of code. Any transformation performed by the compiler while translating a source program must preserve the meaning of the program being compiled. Compiler writers thus have influence over not just the compilers they create, but all the programs that their compilers compile. This leverage makes writing compilers particularly rewarding; however, it also makes compiler development challenging.

1.4.1 Modeling in Compiler Design and Implementation

The study of compilers is mainly a study of how we design the right mathematical models and choose the right algorithms, while balancing the need for generality and power against simplicity and efficiency.

Some of the most fundamental models are finite state machines and regular expressions, which we shall meet in Chapter 3. These models are useful for describing the lexical units of programs: keywords, identifiers, and such, and for describing the algorithms used by the compiler to recognize those units. Also among the most fundamental models are context free grammars, used to describe the syntactic structure of programming languages such as the nesting of parentheses or control constructs. We shall study grammars in Chapter 4. Similarly, trees are an important model for representing the structure of programs and their translation into object code, as we shall see in Chapter 5.

1.4.2 The Science of Code Optimization

The term “optimization” in compiler design refers to the attempts that a compiler makes to produce code that is more efficient than the obvious code. Optimization is thus a misnomer, since there is no way that the code produced by a compiler can be guaranteed to be as fast or faster than any other code that performs the same task.

CHAPTER 1 INTRODUCTION

In modern times the optimization of code that a compiler performs has become both more important and more complex. It is more complex because processor architectures have become more complex, yielding more opportunities to improve the way code executes. It is more important because massively parallel computers require substantial optimization for their performance suffers by orders of magnitude. With the likely prevalence of multicore machines, computers with chips that have large numbers of processors on them, all compilers will have to face the problem of taking advantage of multiprocessor machines.

It is hard, if not impossible, to build a robust compiler out of hacks. Thus, an extensive and useful theory has been built up around the problem of optimizing code. The use of a rigorous mathematical foundation allows us to show that an optimization is correct and that it produces the desirable effect for all possible inputs. We shall see, starting in Chapter 9, how models such as graphs, matrices, and linear programs are necessary if the compiler is to produce well optimized code.

On the other hand, pure theory alone is insufficient. Like many real world problems, there are no perfect answers. In fact, most of the questions that we ask in compiler optimization are undecidable. One of the most important skills in compiler design is the ability to formulate the right problem to solve. We need a good understanding of the behavior of programs to start with and thorough experimentation and evaluation to validate our intuitions.

Compiler optimizations must meet the following design objectives:

The optimization must be correct, that is, preserve the meaning of the compiled program.

The optimization must improve the performance of many programs.

The compilation time must be kept reasonable, and

The engineering effort required must be manageable.

It is impossible to overemphasize the importance of correctness. It is trivial to write a compiler that generates fast code if the generated code need not be correct. Optimizing compilers are so difficult to get right that we dare say that no optimizing compiler is completely error free. Thus, the most important objective in writing a compiler is that it is correct.

The second goal is that the compiler must be effective in improving the performance of many input programs. Normally, performance means the speed of the program execution. Especially in embedded applications, we may also wish to minimize the size of the generated code. And in the case of mobile devices, it is also desirable that the code minimizes power consumption. Typically, the same optimizations that speed up execution time also conserve power. Besides performance, usability aspects such as error reporting and debugging are also important.

Third, we need to keep the compilation time short to support a rapid development and debugging cycle. This requirement has become easier to meet as

1.5 APPLICATIONS OF COMPILER TECHNOLOGY

machines get faster. Often, a program is first developed and debugged without program optimizations. Not only is the compilation time reduced, but more importantly, unoptimized programs are easier to debug, because the optimizations introduced by a compiler often obscure the relationship between the source code and the object code. Turning on optimizations in the compiler sometimes exposes new problems in the source program, thus testing must again be performed on the optimized code. The need for additional testing sometimes deters the use of optimizations in applications, especially if their performance is not critical.

Finally, a compiler is a complex system; we must keep the system simple to assure that the engineering and maintenance costs of the compiler are manageable. There is an infinite number of program optimizations that we could implement, and it takes a nontrivial amount of effort to create a correct and effective optimization. We must prioritize the optimizations, implementing only those that lead to the greatest benefits on source programs encountered in practice.

Thus, in studying compilers, we learn not only how to build a compiler, but also the general methodology of solving complex and open-ended problems. The approach used in compiler development involves both theory and experimentation. We normally start by formulating the problem based on our intuitions on what the important issues are.

1.5 Applications of Compiler Technology

Compiler design is not only about compilers, and many people use the technology learned by studying compilers in school, yet have never, strictly speaking, written even part of a compiler for a major programming language. Compiler technology has other important uses as well. Additionally, compiler design impacts several other areas of computer science. In this section, we review the most important interactions and applications of the technology.

1.5.1 Implementation of High Level Programming Languages

A high-level programming language defines a programming abstraction: the programmer expresses an algorithm using the language, and the compiler must translate that program to the target language. Generally, higher-level programming languages are easier to program in, but are less efficient; that is, the target programs run more slowly. Programmers using a low-level language have more control over a computation and can, in principle, produce more efficient code. Unfortunately, lower-level programs are harder to write and, worse still, less portable, more prone to errors, and harder to maintain. Optimizing compilers include techniques to improve the performance of generated code, thus offsetting the inefficiency introduced by high-level abstractions.

Example 1 2 The **register** keyword in the C programming language is an early example of the interaction between compiler technology and language evolution. When the C language was created in the mid 1970s, it was considered necessary to let a programmer control which program variables reside in registers. This control became unnecessary as effective register allocation techniques were developed, and most modern programs no longer use this language feature.

In fact, programs that use the **register** keyword may lose efficiency because programmers often are not the best judge of very low level matters like register allocation. The optimal choice of register allocation depends greatly on the specifics of a machine architecture. Hardwiring low level resource management decisions like register allocation may in fact hurt performance, especially if the program is run on machines other than the one for which it was written. $\square$

The many shifts in the popular choice of programming languages have been in the direction of increased levels of abstraction. C was the predominant systems programming language of the 80's; many of the new projects started in the 90's chose C++. Java, introduced in 1995, gained popularity quickly in the late 90's. The new programming language features introduced in each round spurred new research in compiler optimization. In the following, we give an overview on the main language features that have stimulated significant advances in compiler technology.

Practically all common programming languages, including C, Fortran, and Cobol, support user defined aggregate data types, such as arrays and structures, and high level control flow, such as loops and procedure invocations. If we just take each high level construct or data access operation and translate it directly to machine code, the result would be very inefficient. A body of compiler optimizations, known as *data flow optimizations*, has been developed to analyze the flow of data through the program and removes redundancies across these constructs. They are effective in generating code that resembles code written by a skilled programmer at a lower level.

Object orientation was first introduced in Simula in 1967, and has been incorporated in languages such as Smalltalk, C++, C#, and Java. The key ideas behind object orientation are

1. Data abstraction and
2. Inheritance of properties

both of which have been found to make programs more modular and easier to maintain. Object oriented programs are different from those written in many other languages, in that they consist of many more, but smaller, procedures, called *methods* in object oriented terms. Thus, compiler optimizations must be able to perform well across the procedural boundaries of the source program. Procedure inlining, which is the replacement of a procedure call by the body of the procedure, is particularly useful here. Optimizations to speed up virtual method dispatches have also been developed.

1.5 APPLICATIONS OF COMPILER TECHNOLOGY

Java has many features that make programming easier many of which have been introduced previously in other languages. The Java language is type safe that is an object cannot be used as an object of an unrelated type. All array accesses are checked to ensure that they lie within the bounds of the array. Java has no pointers and does not allow pointer arithmetic. It has a built in garbage collection facility that automatically frees the memory of variables that are no longer in use. While all these features make programming easier they incur a run time overhead. Compiler optimizations have been developed to reduce the overhead for example by eliminating unnecessary range checks and by allocating objects that are not accessible beyond a procedure on the stack instead of the heap. Effective algorithms also have been developed to minimize the overhead of garbage collection.

In addition Java is designed to support portable and mobile code. Programs are distributed as Java bytecode which must either be interpreted or compiled into native code dynamically that is at run time. Dynamic compilation has also been studied in other contexts where information is extracted dynamically at run time and used to produce better optimized code. In dynamic optimization it is important to minimize the compilation time as it is part of the execution overhead. A common technique used is to only compile and optimize those parts of the program that will be frequently executed.

1.5.2 Optimizations for Computer Architectures

The rapid evolution of computer architectures has also led to an insatiable demand for new compiler technology. Almost all high performance systems take advantage of the same two basic techniques *parallelism* and *memory hierarchies*. Parallelism can be found at several levels at the *instruction level* where multiple operations are executed simultaneously and at the *processor level* where different threads of the same application are run on different processors. Memory hierarchies are a response to the basic limitation that we can build very fast storage or very large storage but not storage that is both fast and large.

Parallelism

All modern microprocessors exploit instruction level parallelism. However this parallelism can be hidden from the programmer. Programs are written as if all instructions were executed in sequence the hardware dynamically checks for dependencies in the sequential instruction stream and issues them in parallel when possible. In some cases the machine includes a hardware scheduler that can change the instruction ordering to increase the parallelism in the program. Whether the hardware reorders the instructions or not compilers can rearrange the instructions to make instruction level parallelism more effective.

Instruction level parallelism can also appear explicitly in the instruction set. VLIW (Very Long Instruction Word) machines have instructions that can issue

CHAPTER 1 INTRODUCTION

multiple operations in parallel. The Intel IA64 is a well known example of such an architecture. All high performance general purpose microprocessors also include instructions that can operate on a vector of data at the same time. Compiler techniques have been developed to generate code automatically for such machines from sequential programs.

Multiprocessors have also become prevalent even personal computers often have multiple processors. Programmers can write multithreaded code for multiprocessors or parallel code can be automatically generated by a compiler from conventional sequential programs. Such a compiler hides from the programmers the details of finding parallelism in a program, distributing the computation across the machine, and minimizing synchronization and communication among the processors. Many scientific computing and engineering applications are computation intensive and can benefit greatly from parallel processing. Parallelization techniques have been developed to translate automatically sequential scientific programs into multiprocessor code.

Memory Hierarchies

A memory hierarchy consists of several levels of storage with different speeds and sizes, with the level closest to the processor being the fastest but smallest. The average memory access time of a program is reduced if most of its accesses are satisfied by the faster levels of the hierarchy. Both parallelism and the existence of a memory hierarchy improve the potential performance of a machine, but they must be harnessed effectively by the compiler to deliver real performance on an application.

Memory hierarchies are found in all machines. A processor usually has a small number of registers consisting of hundreds of bytes, several levels of caches containing kilobytes to megabytes, physical memory containing megabytes to gigabytes, and finally secondary storage that contains gigabytes and beyond. Correspondingly, the speed of accesses between adjacent levels of the hierarchy can differ by two or three orders of magnitude. The performance of a system is often limited not by the speed of the processor but by the performance of the memory subsystem. While compilers traditionally focus on optimizing the processor execution, more emphasis is now placed on making the memory hierarchy more effective.

Using registers effectively is probably the single most important problem in optimizing a program. Unlike registers that have to be managed explicitly in software, caches and physical memories are hidden from the instruction set and are managed by hardware. It has been found that cache management policies implemented by hardware are not effective in some cases, especially in scientific code that has large data structures, arrays, typically. It is possible to improve the effectiveness of the memory hierarchy by changing the layout of the data, or changing the order of instructions accessing the data. We can also change the layout of code to improve the effectiveness of instruction caches.

1.5.3 Design of New Computer Architectures

In the early days of computer architecture design, compilers were developed after the machines were built. That has changed. Since programming in high-level languages is the norm, the performance of a computer system is determined not by its raw speed but also by how well compilers can exploit its features. Thus, in modern computer architecture development, compilers are developed in the processor design stage, and compiled code, running on simulators, is used to evaluate the proposed architectural features.

RISC

One of the best known examples of how compilers influenced the design of computer architecture was the invention of the RISC (Reduced Instruction Set Computer) architecture. Prior to this invention, the trend was to develop progressively complex instruction sets intended to make assembly programming easier; these architectures were known as CISC (Complex Instruction Set Computer). For example, CISC instruction sets include complex memory addressing modes to support data structure accesses and procedure invocation instructions that save registers and pass parameters on the stack.

Compiler optimizations often can reduce these instructions to a small number of simpler operations by eliminating the redundancies across complex instructions. Thus, it is desirable to build simple instruction sets; compilers can use them effectively, and the hardware is much easier to optimize.

Most general-purpose processor architectures, including PowerPC, SPARC, MIPS, Alpha, and PA-RISC, are based on the RISC concept. Although the x86 architecture, the most popular microprocessor, has a CISC instruction set, many of the ideas developed for RISC machines are used in the implementation of the processor itself. Moreover, the most effective way to use a high-performance x86 machine is to use just its simple instructions.

Specialized Architectures

Over the last three decades, many architectural concepts have been proposed. They include data-flow machines, vector machines, VLIW (Very Long Instruction Word) machines, SIMD (Single Instruction Multiple Data) arrays of processors, systolic arrays, multiprocessors with shared memory, and multiprocessors with distributed memory. The development of each of these architectural concepts was accompanied by the research and development of corresponding compiler technology.

Some of these ideas have made their way into the designs of embedded machines. Since entire systems can fit on a single chip, processors need no longer be prepackaged commodity units, but can be tailored to achieve better cost-effectiveness for a particular application. Thus, in contrast to general-purpose processors, where economies of scale have led computer architectures

to converge application specific processors exhibit a diversity of computer architectures. Compiler technology is needed not only to support programming for these architectures but also to evaluate proposed architectural designs.

1.5.4 Program Translations

While we normally think of compiling as a translation from a high level language to the machine level, the same technology can be applied to translate between different kinds of languages. The following are some of the important applications of program translation techniques.

Binary Translation

Compiler technology can be used to translate the binary code for one machine to that of another, allowing a machine to run programs originally compiled for another instruction set. Binary translation technology has been used by various computer companies to increase the availability of software for their machines. In particular, because of the domination of the x86 personal computer market, most software titles are available as x86 code. Binary translators have been developed to convert x86 code into both Alpha and Sparc code. Binary translation was also used by Transmeta Inc. in their implementation of the x86 instruction set. Instead of executing the complex x86 instruction set directly in hardware, the Transmeta Crusoe processor is a VLIW processor that relies on binary translation to convert x86 code into native VLIW code.

Binary translation can also be used to provide backward compatibility. When the processor in the Apple Macintosh was changed from the Motorola MC 68040 to the PowerPC in 1994, binary translation was used to allow PowerPC processors run legacy MC 68040 code.

Hardware Synthesis

Not only is most software written in high level languages, even hardware designs are mostly described in high level hardware description languages like Verilog and VHDL. Very high speed integrated circuit Hardware Description Language. Hardware designs are typically described at the register transfer level, RTL, where variables represent registers and expressions represent combinational logic. Hardware synthesis tools translate RTL descriptions automatically into gates, which are then mapped to transistors and eventually to a physical layout. Unlike compilers for programming languages, these tools often take hours optimizing the circuit. Techniques to translate designs at higher levels, such as the behavior or functional level, also exist.

Database Query Interpreters

Besides specifying software and hardware, languages are useful in many other applications. For example, query languages, especially SQL (Structured Query

1.5 APPLICATIONS OF COMPILER TECHNOLOGY

Language are used to search databases. Database queries consist of predicates containing relational and boolean operators. They can be interpreted or compiled into commands to search a database for records satisfying that predicate.

Compiled Simulation

Simulation is a general technique used in many scientific and engineering disciplines to understand a phenomenon or to validate a design. Inputs to a simulator usually include the description of the design and specific input parameters for that particular simulation run. Simulations can be very expensive. We typically need to simulate many possible design alternatives on many different input sets, and each experiment may take days to complete on a high performance machine. Instead of writing a simulator that interprets the design, it is faster to compile the design to produce machine code that simulates that particular design natively. Compiled simulation can run orders of magnitude faster than an interpreter based approach. Compiled simulation is used in many state of the art tools that simulate designs written in Verilog or VHDL.

1.5.5 Software Productivity Tools

Programs are arguably the most complicated engineering artifacts ever produced; they consist of many many details, every one of which must be correct before the program will work completely. As a result, errors are rampant in programs: errors may crash a system, produce wrong results, render a system vulnerable to security attacks, or even lead to catastrophic failures in critical systems. Testing is the primary technique for locating errors in programs.

An interesting and promising complementary approach is to use data flow analysis to locate errors statically—that is, before the program is run. Data flow analysis can find errors along all the possible execution paths, and not just those exercised by the input data sets, as in the case of program testing. Many of the data flow analysis techniques, originally developed for compiler optimizations, can be used to create tools that assist programmers in their software engineering tasks.

The problem of finding all program errors is undecidable. A data flow analysis may be designed to warn the programmers of all possible statements with a particular category of errors. But if most of these warnings are false alarms, users will not use the tool. Thus, practical error detectors are often neither sound nor complete. That is, they may not find all the errors in the program, and not all errors reported are guaranteed to be real errors. Nonetheless, various static analyses have been developed and shown to be effective in finding errors, such as dereferencing null or freed pointers, in real programs. The fact that error detectors may be unsound makes them significantly different from compiler optimizations. Optimizers must be conservative and cannot alter the semantics of the program under any circumstances.

CHAPTER 1 INTRODUCTION

In the balance of this section we shall mention several ways in which program analysis building upon techniques originally developed to optimize code in compilers have improved software productivity. Of special importance are techniques that detect statically when a program might have a security vulnerability.

Type Checking

Type checking is an effective and well established technique to catch inconsistencies in programs. It can be used to catch errors for example where an operation is applied to the wrong type of object or if parameters passed to a procedure do not match the signature of the procedure. Program analysis can go beyond finding type errors by analyzing the flow of data through a program. For example if a pointer is assigned `null` and then immediately dereferenced the program is clearly in error.

The same technology can be used to catch a variety of security holes in which an attacker supplies a string or other data that is used carelessly by the program. A user supplied string can be labeled with a type `dangerous`. If this string is not checked for proper format then it remains `dangerous` and if a string of this type is able to influence the control flow of the code at some point in the program then there is a potential security flaw.

Bounds Checking

It is easier to make mistakes when programming in a lower level language than a higher level one. For example many security breaches in systems are caused by buffer over flows in programs written in C. Because C does not have array bounds checks it is up to the user to ensure that the arrays are not accessed out of bounds. Failing to check that the data supplied by the user can overflow a buffer the program may be tricked into storing user data outside of the buffer. An attacker can manipulate the input data that causes the program to misbehave and compromise the security of the system. Techniques have been developed to find buffer over flows in programs but with limited success.

Had the program been written in a safe language that includes automatic range checking this problem would not have occurred. The same data flow analysis that is used to eliminate redundant range checks can also be used to locate buffer over flows. The major difference however is that failing to eliminate a range check would only result in a small run time cost while failing to identify a potential buffer overflow may compromise the security of the system. Thus while it is adequate to use simple techniques to optimize range checks sophisticated analyses such as tracking the values of pointers across procedures are needed to get high quality results in error detection tools.

Memory Management Tools

Garbage collection is another excellent example of the tradeoff between efficiency and a combination of ease of programming and software reliability. Automatic memory management obliterates all memory management errors (e.g., memory leaks) which are a major source of problems in C and C++ programs. Various tools have been developed to help programmers find memory management errors. For example, Purify is a widely used tool that dynamically catches memory management errors as they occur. Tools that help identify some of these problems statically have also been developed.

1 6 Programming Language Basics

In this section, we shall cover the most important terminology and distinctions that appear in the study of programming languages. It is not our purpose to cover all concepts or all the popular programming languages. We assume that the reader is familiar with at least one of C, C++, C#, or Java, and may have encountered other languages as well.

1 6 1 The Static-Dynamic Distinction

Among the most important issues that we face when designing a compiler for a language is what decisions can the compiler make about a program. If a language uses a policy that allows the compiler to decide an issue, then we say that the language uses a *static* policy or that the issue can be decided at *compile time*. On the other hand, a policy that only allows a decision to be made when we execute the program is said to be a *dynamic policy* or to require a decision at *run time*.

One issue on which we shall concentrate is the scope of declarations. The *scope* of a declaration of x is the region of the program in which uses of x refer to this declaration. A language uses *static scope* or *lexical scope* if it is possible to determine the scope of a declaration by looking only at the program. Otherwise, the language uses *dynamic scope*. With dynamic scope, as the program runs, the same use of x could refer to any of several different declarations of x .

Most languages, such as C and Java, use static scope. We shall discuss static scoping in Section 1 6 3.

Example 1 3 As another example of the static-dynamic distinction, consider the use of the term “static” as it applies to data in a Java class declaration. In Java, a variable is a name for a location in memory used to hold a data value. Here, “static” refers not to the scope of the variable, but rather to the ability of the compiler to determine the location in memory where the declared variable can be found. A declaration like

```
public static int x
```

kind of identifier may be read from the symbol table by the lexical analyzer to assist it in determining the proper token it must pass to the parser

These interactions are suggested in Fig. 3.1. Commonly, the interaction is implemented by having the parser call the lexical analyzer. The call, suggested by the *getNextToken* command, causes the lexical analyzer to read characters from its input until it can identify the next lexeme and produce for it the next token, which it returns to the parser.

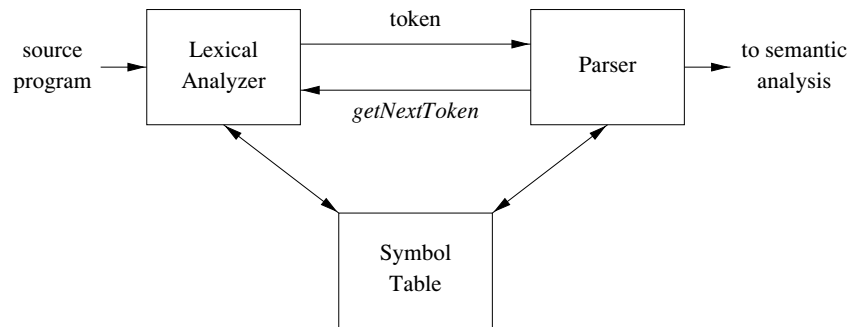


Figure 3.1 Interactions between the lexical analyzer and the parser

Since the lexical analyzer is the part of the compiler that reads the source text, it may perform certain other tasks besides identification of lexemes. One such task is stripping out comments and *whitespace* (blank, newline, tab, and perhaps other characters that are used to separate tokens in the input). Another task is correlating error messages generated by the compiler with the source program. For instance, the lexical analyzer may keep track of the number of newline characters seen, so it can associate a line number with each error message. In some compilers, the lexical analyzer makes a copy of the source program with the error messages inserted at the appropriate positions. If the source program uses a macro preprocessor, the expansion of macros may also be performed by the lexical analyzer.

Sometimes, lexical analyzers are divided into a cascade of two processes:

- a. *Scanning* consists of the simple processes that do not require tokenization of the input, such as deletion of comments and compaction of consecutive whitespace characters into one.
- b. *Lexical analysis proper* is the more complex portion, which produces tokens from the output of the scanner.

3.1.1 Lexical Analysis Versus Parsing

There are a number of reasons why the analysis portion of a compiler is normally separated into lexical analysis and parsing (syntax analysis) phases.

3.1 THE ROLE OF THE LEXICAL ANALYZER

1. Simplicity of design is the most important consideration. The separation of lexical and syntactic analysis often allows us to simplify at least one of these tasks. For example, a parser that had to deal with comments and whitespace as syntactic units would be considerably more complex than one that can assume comments and whitespace have already been removed by the lexical analyzer. If we are designing a new language separating lexical and syntactic concerns can lead to a cleaner overall language design.
2. Compiler efficiency is improved. A separate lexical analyzer allows us to apply specialized techniques that serve only the lexical task, not the job of parsing. In addition, specialized buffering techniques for reading input characters can speed up the compiler significantly.
3. Compiler portability is enhanced. Input device specific peculiarities can be restricted to the lexical analyzer.

3.1.2 Tokens, Patterns, and Lexemes

When discussing lexical analysis, we use three related but distinct terms:

A *token* is a pair consisting of a token name and an optional attribute value. The token name is an abstract symbol representing a kind of lexical unit, e.g., a particular keyword, or a sequence of input characters denoting an identifier. The token names are the input symbols that the parser processes. In what follows, we shall generally write the name of a token in boldface. We will often refer to a token by its token name.

A *pattern* is a description of the form that the lexemes of a token may take. In the case of a keyword as a token, the pattern is just the sequence of characters that form the keyword. For identifiers and some other tokens, the pattern is a more complex structure that is *matched* by many strings.

A *lexeme* is a sequence of characters in the source program that matches the pattern for a token and is identified by the lexical analyzer as an instance of that token.

Example 3.1 Figure 3.2 gives some typical tokens, their informally described patterns, and some sample lexemes. To see how these concepts are used in practice, in the C statement

```
printf "Total: %d\n", score;
```

both `printf` and `score` are lexemes matching the pattern for token **id**, and `Total: %d\n` is a lexeme matching **literal**. □

In many programming languages, the following classes cover most or all of the tokens:

TOKEN	INFORMAL DESCRIPTION	SAMPLE LEXEMES
if	characters <code>i f</code>	<code>if</code>
else	characters <code>e l s e</code>	<code>else</code>
comparison	<code>or or or or or</code>	
id	letter followed by letters and digits	<code>pi score D2</code>
number	any numeric constant	<code>3 14159 0 6 02e23</code>
literal	anything but <code> </code> surrounded by <code> </code> <code>s</code>	<code>core dumped</code>

Figure 3.2 Examples of tokens

- 1 One token for each keyword. The pattern for a keyword is the same as the keyword itself.
- 2 Tokens for the operators, either individually or in classes such as the token `comparison` mentioned in Fig. 3.2.
- 3 One token representing all identifiers.
- 4 One or more tokens representing constants, such as numbers and literal strings.
- 5 Tokens for each punctuation symbol, such as left and right parentheses, comma, and semicolon.

3.1.3 Attributes for Tokens

When more than one lexeme can match a pattern, the lexical analyzer must provide the subsequent compiler phases additional information about the particular lexeme that matched. For example, the pattern for token **number** matches both 0 and 1, but it is extremely important for the code generator to know which lexeme was found in the source program. Thus, in many cases the lexical analyzer returns to the parser not only a token name, but an attribute value that describes the lexeme represented by the token: the token name influences parsing decisions, while the attribute value influences translation of tokens after the parse.

We shall assume that tokens have at most one associated attribute, although this attribute may have a structure that combines several pieces of information. The most important example is the token **id**, where we need to associate with the token a great deal of information. Normally, information about an identifier, e.g., its lexeme, its type, and the location at which it is first found, in case an error message about that identifier must be issued, is kept in the symbol table. Thus, the appropriate attribute value for an identifier is a pointer to the symbol table entry for that identifier.

Tricky Problems When Recognizing Tokens

Usually given the pattern describing the lexemes of a token it is relatively simple to recognize matching lexemes when they occur on the input. However, in some languages it is not immediately apparent when we have seen an instance of a lexeme corresponding to a token. The following example is taken from Fortran in the fixed format still allowed in Fortran 90. In the statement

```
DO 5 I 1 25
```

it is not apparent that the first lexeme is `D05I` an instance of the identifier token until we see the dot following the 1. Note that blanks in fixed format Fortran are ignored, an archaic convention. Had we seen a comma instead of the dot we would have had a `do` statement

```
DO 5 I 1 25
```

in which the first lexeme is the keyword `DO`

Example 3.2 The token names and associated attribute values for the Fortran statement

```
E M C 2
```

are written below as a sequence of pairs

```
id pointer to symbol table entry for E
assign_op
id pointer to symbol table entry for M
mult_op
id pointer to symbol table entry for C
exp_op
number integer value 2
```

Note that in certain pairs, especially operators, punctuation, and keywords, there is no need for an attribute value. In this example, the token **number** has been given an integer valued attribute. In practice, a typical compiler would instead store a character string representing the constant and use as an attribute value for **number** a pointer to that string. □

3.1.4 Lexical Errors

It is hard for a lexical analyzer to tell, without the aid of other components, that there is a source code error. For instance, if the string `fi` is encountered for the first time in a C program in the context

```
fi a f x
```

a lexical analyzer cannot tell whether `fi` is a misspelling of the keyword `if` or an undeclared function identifier. Since `fi` is a valid lexeme for the token `id`, the lexical analyzer must return the token `id` to the parser and let some other phase of the compiler—probably the parser in this case—handle an error due to transposition of the letters.

However, suppose a situation arises in which the lexical analyzer is unable to proceed because none of the patterns for tokens matches any prefix of the remaining input. The simplest recovery strategy is panic mode recovery. We delete successive characters from the remaining input until the lexical analyzer can find a well formed token at the beginning of what input is left. This recovery technique may confuse the parser, but in an interactive computing environment it may be quite adequate.

Other possible error recovery actions are

- 1 Delete one character from the remaining input
- 2 Insert a missing character into the remaining input
- 3 Replace a character by another character
- 4 Transpose two adjacent characters

Transformations like these may be tried in an attempt to repair the input. The simplest such strategy is to see whether a prefix of the remaining input can be transformed into a valid lexeme by a single transformation. This strategy makes sense, since in practice most lexical errors involve a single character. A more general correction strategy is to find the smallest number of transformations needed to convert the source program into one that consists only of valid lexemes, but this approach is considered too expensive in practice to be worth the effort.

3.1.5 Exercises for Section 3.1

Exercise 3.1.1 Divide the following C program

```
float limitedSquare(x) float x
    returns x squared but never more than 100
return x < 10.0 ? x : 100.0
```

into appropriate lexemes, using the discussion of Section 3.1.2 as a guide. Which lexemes should get associated lexical values? What should those values be?

Exercise 3.1.2 Tagged languages like HTML or XML are different from conventional programming languages in that the punctuation tags are either very numerous, as in HTML, or a user definable set, as in XML. Further, tags can often have parameters. Suggest how to divide the following HTML document

3.2 INPUT BUFFERING

```
Here is a photo of B my house B
P IMG SRC house gif BR
See A HREF morePix html More Pictures A if you
liked that one P
```

into appropriate lexemes. Which lexemes should get associated lexical values and what should those values be?

3.2 Input Buffering

Before discussing the problem of recognizing lexemes in the input, let us examine some ways that the simple but important task of reading the source program can be speeded. This task is made difficult by the fact that we often have to look one or more characters beyond the next lexeme before we can be sure we have the right lexeme. The box on “Tricky Problems When Recognizing Tokens” in Section 3.1 gave an extreme example, but there are many situations where we need to look at least one additional character ahead. For instance, we cannot be sure we’ve seen the end of an identifier until we see a character that is not a letter or digit, and therefore is not part of the lexeme for **id**. In C, single character operators like `+` or `-` could also be the beginning of a two character operator like `++` or `--`. Thus, we shall introduce a two buffer scheme that handles large lookaheads safely. We then consider an improvement involving “sentinels” that saves time checking for the ends of buffers.

3.2.1 Buffer Pairs

Because of the amount of time taken to process characters and the large number of characters that must be processed during the compilation of a large source program, specialized buffering techniques have been developed to reduce the amount of overhead required to process a single input character. An important scheme involves two buffers that are alternately reloaded, as suggested in Fig. 3.3.

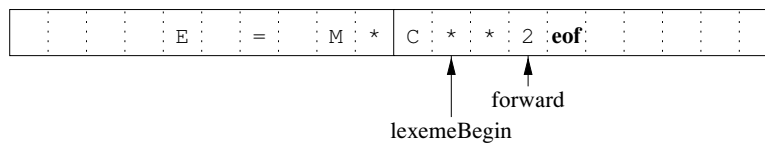


Figure 3.3 Using a pair of input buffers

Each buffer is of the same size N , and N is usually the size of a disk block (e.g., 4096 bytes). Using one system read command, we can read N characters into a buffer rather than using one system call per character. If fewer than N characters remain in the input file, then a special character, represented by **eof**,

marks the end of the source file and is different from any possible character of the source program

Two pointers to the input are maintained

- 1 Pointer `lexemeBegin` marks the beginning of the current lexeme whose extent we are attempting to determine
- 2 Pointer `forward` scans ahead until a pattern match is found the exact strategy whereby this determination is made will be covered in the balance of this chapter

Once the next lexeme is determined `forward` is set to the character at its right end Then after the lexeme is recorded as an attribute value of a token returned to the parser `lexemeBegin` is set to the character immediately after the lexeme just found In Fig 3.3 we see `forward` has passed the end of the next lexeme

the Fortran exponentiation operator and must be retracted one position to its left

Advancing `forward` requires that we first test whether we have reached the end of one of the buffers and if so we must reload the other buffer from the input and move `forward` to the beginning of the newly loaded buffer As long as we never need to look so far ahead of the actual lexeme that the sum of the lexeme's length plus the distance we look ahead is greater than N we shall never overwrite the lexeme in its buffer before determining it

3.2.2 Sentinels

If we use the scheme of Section 3.2.1 as described we must check each time we advance `forward` that we have not moved off one of the buffers if we do then we must also reload the other buffer Thus for each character read we make two tests one for the end of the buffer and one to determine what character is read the latter may be a multiway branch We can combine the buffer end test with the test for the current character if we extend each buffer to hold a *sentinel* character at the end The sentinel is a special character that cannot be part of the source program and a natural choice is the character `eof`

Figure 3.4 shows the same arrangement as Fig 3.3 but with the sentinels added Note that `eof` retains its use as a marker for the end of the entire input Any `eof` that appears other than at the end of a buffer means that the input is at an end Figure 3.5 summarizes the algorithm for advancing `forward` Notice how the first test which can be part of a multiway branch based on the character pointed to by `forward` is the only test we make except in the case where we actually are at the end of a buffer or the end of the input

3.3 Specification of Tokens

Regular expressions are an important notation for specifying lexeme patterns While they cannot express all possible patterns they are very effective in spec

Can We Run Out of Buffer Space

In most modern languages, lexemes are short, and one or two characters of lookahead is sufficient. Thus a buffer size N in the thousands is ample, and the double buffer scheme of Section 3.2.1 works without problem. However, there are some risks. For example, if character strings can be very long, extending over many lines, then we could face the possibility that a lexeme is longer than N . To avoid problems with long character strings, we can treat them as a concatenation of components, one from each line over which the string is written. For instance, in Java it is conventional to represent long strings by writing a piece on each line and concatenating pieces with a `\n` operator at the end of each piece.

A more difficult problem occurs when arbitrarily long lookahead may be needed. For example, some languages like PL/I do not treat keywords as *reserved*; that is, you can use identifiers with the same name as a keyword like `DECLARE`. If the lexical analyzer is presented with text of a PL/I program that begins `DECLARE ARG1 ARG2`, it cannot be sure whether `DECLARE` is a keyword, and `ARG1` and so on are variables being declared, or whether `DECLARE` is a procedure name with its arguments. For this reason, modern languages tend to reserve their keywords. However, if not, one can treat a keyword like `DECLARE` as an ambiguous identifier, and let the parser resolve the issue, perhaps in conjunction with symbol table lookup.

ifying those types of patterns that we actually need for tokens. In this section we shall study the formal notation for regular expressions, and in Section 3.5 we shall see how these expressions are used in a lexical analyzer generator. Then, Section 3.7 shows how to build the lexical analyzer by converting regular expressions to automata that perform the recognition of the specified tokens.

3.3.1 Strings and Languages

An *alphabet* is any finite set of symbols. Typical examples of symbols are letters, digits, and punctuation. The set $\{0, 1\}$ is the *binary alphabet*. ASCII is an important example of an alphabet; it is used in many software systems. Uni-

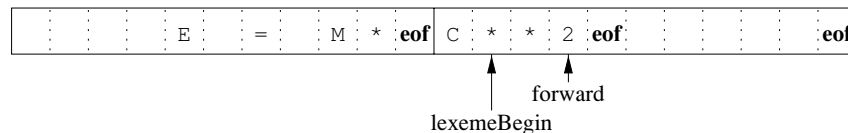


Figure 3.4 Sentinels at the end of each buffer

```

switch forward {
  case eof
    if forward is at end of first buffer {
      reload second buffer
      forward = beginning of second buffer
    }
    else if forward is at end of second buffer {
      reload first buffer
      forward = beginning of first buffer
    }
    else    eof within a buffer marks the end of input
            terminate lexical analysis
    break
  Cases for the other characters
}

```

Figure 3.5 Lookahead code with sentinels

Implementing Multiway Branches

We might imagine that the switch in Fig. 3.5 requires many steps to execute and that placing the case **eof** first is not a wise choice. Actually it doesn't matter in what order we list the cases for each character. In practice a multiway branch depending on the input character is made in one step by jumping to an address found in an array of addresses indexed by characters.

code which includes approximately 100 000 characters from alphabets around the world is another important example of an alphabet.

A *string* over an alphabet is a finite sequence of symbols drawn from that alphabet. In language theory the terms *sentence* and *word* are often used as synonyms for *string*. The length of a string s usually written $|s|$ is the number of occurrences of symbols in s . For example **banana** is a string of length six. The *empty string* denoted ϵ is the string of length zero.

A *language* is any countable set of strings over some fixed alphabet. This definition is very broad. Abstract languages like the *empty set* or $\{\epsilon\}$ the set containing only the empty string are languages under this definition. So too are the set of all syntactically well formed C programs and the set of all grammatically correct English sentences although the latter two languages are difficult to specify exactly. Note that the definition of *language* does not require that any meaning be ascribed to the strings in the language. Methods for defining the meaning of strings are discussed in Chapter 5.

Terms for Parts of Strings

The following string related terms are commonly used

- 1 A *prefix* of string s is any string obtained by removing zero or more symbols from the end of s . For example `ban` `banana` and are prefixes of `banana`.
- 2 A *suffix* of string s is any string obtained by removing zero or more symbols from the beginning of s . For example `nana` `banana` and are suffixes of `banana`.
- 3 A *substring* of s is obtained by deleting any prefix and any suffix from s . For instance `banana` `nan` and are substrings of `banana`.
- 4 The *proper* prefixes, suffixes and substrings of a string s are those prefixes, suffixes and substrings respectively of s that are not or not equal to s itself.
- 5 A *subsequence* of s is any string formed by deleting zero or more not necessarily consecutive positions of s . For example `baan` is a subsequence of `banana`.

If x and y are strings then the *concatenation* of x and y denoted xy is the string formed by appending y to x . For example if $x = \text{dog}$ and $y = \text{house}$ then $xy = \text{doghouse}$. The empty string is the identity under concatenation that is for any string s $s \cdot \epsilon = s = \epsilon \cdot s$.

If we think of concatenation as a product we can define the exponentiation of strings as follows. Define s^0 to be ϵ and for all $i \geq 0$ define s^i to be $s^{i-1}s$. Since $s = s \cdot \epsilon$ it follows that $s^1 = s$. Then $s^2 = ss$, $s^3 = sss$ and so on.

3.3.2 Operations on Languages

In lexical analysis the most important operations on languages are union, concatenation and closure which are defined formally in Fig. 3.6. Union is the familiar operation on sets. The concatenation of languages is all strings formed by taking a string from the first language and a string from the second language in all possible ways and concatenating them. The *Kleene closure* of a language L denoted L^* is the set of strings you get by concatenating L zero or more times. Note that L^0 the concatenation of L zero times is defined to be $\{\epsilon\}$ and inductively L^i is $L^{i-1}L$. Finally the positive closure denoted L^+ is the same as the Kleene closure but without the term L^0 . That is ϵ will not be in L^+ unless it is in L itself.

OPERATION	DEFINITION AND NOTATION
<i>Union</i> of L and M	$L \cup M \quad \{s \mid s \text{ is in } L \text{ or } s \text{ is in } M\}$
<i>Concatenation</i> of L and M	$LM \quad \{st \mid s \text{ is in } L \text{ and } t \text{ is in } M\}$
<i>Kleene closure</i> of L	$L^* \quad \bigcup_{i=0}^{\infty} L^i$
<i>Positive closure</i> of L	$L^+ \quad \bigcup_{i=1}^{\infty} L^i$

Figure 3.6 Definitions of operations on languages

Example 3.3 Let L be the set of letters $\{A, B, \dots, Z, a, b, \dots, z\}$ and let D be the set of digits $\{0, 1, \dots, 9\}$. We may think of L and D in two essentially equivalent ways. One way is that L and D are respectively the alphabets of uppercase and lowercase letters and of digits. The second way is that L and D are languages—all of whose strings happen to be of length one. Here are some other languages that can be constructed from languages L and D using the operators of Fig. 3.6.

1. $L \cup D$ is the set of letters and digits—strictly speaking the language with 62 strings of length one—each of which string is either one letter or one digit.
2. LD is the set of 520 strings of length two—each consisting of one letter followed by one digit.
3. L^4 is the set of all 4-letter strings.
4. L^* is the set of all strings of letters—including the empty string.
5. $L L^* D^*$ is the set of all strings of letters and digits beginning with a letter.
6. D^+ is the set of all strings of one or more digits.

□

3.3.3 Regular Expressions

Suppose we wanted to describe the set of valid C identifiers. It is almost exactly the language described in item 5 above—the only difference is that the underscore is included among the letters.

In Example 3.3 we were able to describe identifiers by giving names to sets of letters and digits and using the language operators union, concatenation, and closure. This process is so useful that a notation called *regular expressions* has come into common use for describing all the languages that can be built from these operators applied to the symbols of some alphabet. In this notation if *letter_* is established to stand for any letter or the underscore and *digit* is

3.3 SPECIFICATION OF TOKENS

established to stand for any digit then we could describe the language of C identifiers by

$$letter_letter_ | digit$$

The vertical bar above means union the parentheses are used to group subexpressions the star means zero or more occurrences of and the juxtaposition of *letter_* with the remainder of the expression signifies concatenation

The regular expressions are built recursively out of smaller regular expressions using the rules described below Each regular expression *r* denotes a language L_r which is also defined recursively from the languages denoted by *r*'s subexpressions Here are the rules that define the regular expressions over some alphabet and the languages that those expressions denote

BASIS There are two rules that form the basis

- 1 *ε* is a regular expression and L_{ϵ} is $\{\}$ that is the language whose sole member is the empty string
- 2 If *a* is a symbol in then ***a*** is a regular expression and $L_{\mathbf{a}} = \{a\}$ that is the language with one string of length one with *a* in its one position Note that by convention we use italics for symbols and boldface for their corresponding regular expression¹

INDUCTION There are four parts to the induction whereby larger regular expressions are built from smaller ones Suppose *r* and *s* are regular expressions denoting languages L_r and L_s respectively

- 1 *r* | *s* is a regular expression denoting the language $L_r \cup L_s$
- 2 *r* *s* is a regular expression denoting the language $L_r L_s$
- 3 *r** is a regular expression denoting L_r^*
- 4 (*r*) is a regular expression denoting L_r This last rule says that we can add additional pairs of parentheses around expressions without changing the language they denote

As defined regular expressions often contain unnecessary pairs of parentheses We may drop certain pairs of parentheses if we adopt the conventions that

- a The unary operator * has highest precedence and is left associative
- b Concatenation has second highest precedence and is left associative

¹However when talking about specific characters from the ASCII character set we shall generally use teletype font for both the character and its regular expression

c $|$ has lowest precedence and is left associative

Under these conventions for example we may replace the regular expression $\mathbf{a} \mid \mathbf{b} \mid \mathbf{c}$ by $\mathbf{a} \mid \mathbf{b} \mid \mathbf{c}$. Both expressions denote the set of strings that are either a single a or are zero or more b 's followed by one c .

Example 3.4 Let $\Sigma = \{a, b\}$

- 1 The regular expression $\mathbf{a} \mid \mathbf{b}$ denotes the language $\{a, b\}$
- 2 $\mathbf{a} \mid \mathbf{b} \mid \mathbf{a} \mid \mathbf{b}$ denotes $\{aa, ab, ba, bb\}$ the language of all strings of length two over the alphabet Σ . Another regular expression for the same language is $\mathbf{aa} \mid \mathbf{ab} \mid \mathbf{ba} \mid \mathbf{bb}$
- 3 $\mathbf{a}^*$ denotes the language consisting of all strings of zero or more a 's that is $\{\epsilon, a, aa, aaa, \dots\}$
- 4 $\mathbf{a}^* \mid \mathbf{b}^*$ denotes the set of all strings consisting of zero or more instances of a or b that is all strings of a 's and b 's $\{\epsilon, a, b, aa, ab, ba, bb, aaa, \dots\}$. Another regular expression for the same language is $\mathbf{a}^* \mid \mathbf{b}^*$
- 5 $\mathbf{a}^* \mid \mathbf{a} \mid \mathbf{b}$ denotes the language $\{a, b, ab, aab, aaab, \dots\}$ that is the string a and all strings consisting of zero or more a 's and ending in b .

□

A language that can be defined by a regular expression is called a *regular set*. If two regular expressions r and s denote the same regular set we say they are *equivalent* and write $r \equiv s$. For instance $\mathbf{a} \mid \mathbf{b} \equiv \mathbf{b} \mid \mathbf{a}$. There are a number of algebraic laws for regular expressions each law asserts that expressions of two different forms are equivalent. Figure 3.7 shows some of the algebraic laws that hold for arbitrary regular expressions r , s , and t .

LAW	DESCRIPTION
$r \mid s \equiv s \mid r$	$ $ is commutative
$r \mid (s \mid t) \equiv (r \mid s) \mid t$	$ $ is associative
$r \mid st \equiv rs \mid t$	Concatenation is associative
$r \mid s \mid t \equiv rs \mid rt \equiv s \mid tr \equiv sr \mid tr$	Concatenation distributes over $ $
$r \mid \epsilon \equiv r$	ϵ is the identity for concatenation
$r \mid r^*$	r^* is guaranteed in a closure
$r \mid r \equiv r$	$ $ is idempotent

Figure 3.7 Algebraic laws for regular expressions

3.3 SPECIFICATION OF TOKENS

3.3.4 Regular Definitions

For notational convenience we may wish to give names to certain regular expressions and use those names in subsequent expressions as if the names were themselves symbols. If Σ is an alphabet of basic symbols then a *regular definition* is a sequence of definitions of the form

$$\begin{array}{ll} d_1 & r_1 \\ d_2 & r_2 \\ & \vdots \\ d_n & r_n \end{array}$$

where

- 1 Each d_i is a new symbol not in Σ and not the same as any other of the d 's and
- 2 Each r_i is a regular expression over the alphabet $\{\Sigma \cup \{d_1, d_2, \dots, d_{i-1}\}\}$

By restricting r_i to Σ and the previously defined d 's we avoid recursive definitions and we can construct a regular expression over Σ alone for each r_i . We do so by first replacing uses of d_1 in r_2 which cannot use any of the d 's except for d_1 then replacing uses of d_1 and d_2 in r_3 by r_1 and the substituted r_2 and so on. Finally in r_n we replace each d_i for $i = 1, 2, \dots, n-1$ by the substituted version of r_i each of which has only symbols of Σ .

Example 3.5 C identifiers are strings of letters, digits, and underscores. Here is a regular definition for the language of C identifiers. We shall conventionally use italics for the symbols defined in regular definitions.

$$\begin{array}{ll} \textit{letter_} & A \mid B \mid \dots \mid Z \mid a \mid b \mid \dots \mid z \mid _ \\ \textit{digit} & 0 \mid 1 \mid \dots \mid 9 \\ \textit{id} & \textit{letter_} \mid \textit{letter_} \mid \textit{digit} \end{array}$$

□

Example 3.6 Unsigned numbers (integer or floating point) are strings such as 5280, 0.01234, 6.336E4, or 1.89E-4. The regular definition

$$\begin{array}{ll} \textit{digit} & 0 \mid 1 \mid \dots \mid 9 \\ \textit{digits} & \textit{digit} \mid \textit{digit} \textit{digit} \\ \textit{optionalFraction} & \textit{digits} \mid \\ \textit{optionalExponent} & E \mid \mid \textit{digits} \mid \\ \textit{number} & \textit{digits} \mid \textit{optionalFraction} \mid \textit{optionalExponent} \end{array}$$

is a precise specification for this set of strings. That is, an *optionalFraction* is either a decimal point (dot) followed by one or more digits, or it is missing (the empty string). An *optionalExponent* if not missing is the letter E followed by an optional + or - sign followed by one or more digits. Note that at least one digit must follow the dot, so *number* does not match 1. but does match 1.0.

□

3.3.5 Extensions of Regular Expressions

Since Kleene introduced regular expressions with the basic operators for union, concatenation, and Kleene closure in the 1950s, many extensions have been added to regular expressions to enhance their ability to specify string patterns. Here we mention a few notational extensions that were first incorporated into Unix utilities such as **Lex** that are particularly useful in the specification of lexical analyzers. The references to this chapter contain a discussion of some regular expression variants in use today.

- 1 *One or more instances* The unary postfix operator $|$ represents the positive closure of a regular expression and its language. That is, if r is a regular expression, then $r|$ denotes the language $L(r|)$. The operator $|$ has the same precedence and associativity as the operator $*$. Two useful algebraic laws $r| = r|$ and $r| = rr^*$ relate the Kleene closure and positive closure.
- 2 *Zero or one instance* The unary postfix operator $|?$ means "zero or one occurrence." That is, $r|?$ is equivalent to $r|$ or, put another way, $L(r|?) = L(r) \cup \{\epsilon\}$. The operator $|?$ has the same precedence and associativity as $|$ and $*$.
- 3 *Character classes* A regular expression $a_1|a_2|\dots|a_n$, where the a_i 's are each symbols of the alphabet, can be replaced by the shorthand $a_1a_2\dots a_n$. More importantly, when $a_1a_2\dots a_n$ form a logical sequence (e.g., consecutive uppercase letters, lowercase letters, or digits), we can replace them by a_1a_n , that is, just the first and last separated by a hyphen. Thus **abc** is shorthand for **a|b|c** and **abcdefghijklmnopqrstuvwxyz** is shorthand for **a|b|...|z**.

Example 3.7 Using these shorthands, we can rewrite the regular definition of Example 3.5 as

<i>letter_</i>	A-Za-z_
<i>digit</i>	0-9
<i>id</i>	letter_ letter_ digit

The regular definition of Example 3.6 can also be simplified:

<i>digit</i>	0-9
<i>digits</i>	digit
<i>number</i>	digits E digits

□

3.3 SPECIFICATION OF TOKENS

3.3.6 Exercises for Section 3.3

Exercise 3.3.1 Consult the language reference manuals to determine *i* the sets of characters that form the input alphabet (excluding those that may only appear in character strings or comments) *ii* the lexical form of numerical constants and *iii* the lexical form of identifiers for each of the following languages: a C b C c C d Fortran e Java f Lisp g SQL

Exercise 3.3.2 Describe the languages denoted by the following regular expressions

- a $a|b$
- b $|a$
- c $a|b$ $a|b$ $a|b$ $a|b$
- d a ba ba ba
- e $aa|bb$ $ab|ba$ $aa|bb$ $ab|ba$ $aa|bb$

Exercise 3.3.3 In a string of length n how many of the following are there

- a Prefixes
- b Suffixes
- c Proper prefixes
- d Substrings
- e Subsequences

Exercise 3.3.4 Most languages are *case sensitive* so keywords can be written only one way and the regular expressions describing their lexemes are very simple. However, some languages like SQL are *case insensitive* so a keyword can be written either in lowercase or in uppercase or in any mixture of cases. Thus, the SQL keyword `SELECT` can also be written `select` `Select` or `sElEcT` for instance. Show how to write a regular expression for a keyword in a case insensitive language. Illustrate the idea by writing the expression for `select` in SQL.

Exercise 3.3.5 Write regular definitions for the following languages

- a All strings of lowercase letters that contain the five vowels in order
- b All strings of lowercase letters in which the letters are in ascending lexicographic order
- c Comments consisting of a string surrounded by `/*` and `*/` without an intervening `*/` unless it is inside double quotes

CHAPTER 3 LEXICAL ANALYSIS

- d All strings of digits with no repeated digits *Hint* Try this problem first with a few digits such as {0 1 2}
- e All strings of digits with at most one repeated digit
- f All strings of *a* s and *b* s with an even number of *a* s and an odd number of *b* s
- g The set of Chess moves in the informal notation such as *p k4* or *kbp qn*
- h All strings of *a* s and *b* s that do not contain the substring *abb*
- i All strings of *a* s and *b* s that do not contain the subsequence *abb*

Exercise 3 3 6 Write character classes for the following sets of characters

- a The first ten letters up to *j* in either upper or lower case
- b The lowercase consonants
- c The digits in a hexadecimal number choose either upper or lower case for the digits above 9
- d The characters that can appear at the end of a legitimate English sentence e.g. exclamation point

The following exercises up to and including Exercise 3 3 10 discuss the extended regular expression notation from *Lex* the lexical analyzer generator that we shall discuss extensively in Section 3 5 The extended notation is listed in Fig 3 8

Exercise 3 3 7 Note that these regular expressions give all of the following symbols *operator characters* a special meaning

Their special meaning must be turned off if they are needed to represent themselves in a character string We can do so by quoting the character within a string of length one or more e.g. the regular expression `"\" matches the string`

We can also get the literal meaning of an operator character by preceding it by a backslash Thus the regular expression `\" also matches the string`

Write a regular expression that matches the string

Exercise 3 3 8 In *Lex* a *complemented character class* represents any character except the ones listed in the character class We denote a complemented class by using `^` as the first character this symbol caret is not itself part of the class being complemented unless it is listed within the class itself Thus

`A-Za-z` matches any character that is not an uppercase or lowercase letter and `^` represents any character but the caret or newline since newline cannot be in any character class Show that for every regular expression with complemented character classes there is an equivalent regular expression without complemented character classes

3.3 SPECIFICATION OF TOKENS

EXPRESSION	MATCHES	EXAMPLE
c	the one non operator character c	<code>a</code>
$\backslash c$	character c literally	
s	string s literally	
	any character but newline	<code>a b</code>
	beginning of a line	<code>abc</code>
	end of a line	<code>abc</code>
s	any one of the characters in string s	<code>abc</code>
$\sim s$	any one character not in string s	<code>abc</code>
r^*	zero or more strings matching r	<code>a</code>
r^+	one or more strings matching r	<code>a</code>
$r?$	zero or one r	<code>a</code>
$r\{m\ n\}$	between m and n occurrences of r	<code>a{1 5}</code>
$r_1 r_2$	an r_1 followed by an r_2	<code>ab</code>
$r_1 \mid r_2$	an r_1 or an r_2	<code>a b</code>
$\dot{r}$	same as r	<code>a b</code>
$r_1 \ r_2$	r_1 when followed by r_2	<code>abc 123</code>

Figure 3.8 Lex regular expressions

Exercise 3.3.9 The regular expression $r\{m\ n\}$ matches from m to n occurrences of the pattern r . For example `a{1 5}` matches a string of one to five `a`'s. Show that for every regular expression containing repetition operators of this form, there is an equivalent regular expression without repetition operators.

Exercise 3.3.10 The operator `^` matches the left end of a line and `$` matches the right end of a line. The operator `.` is also used to introduce complemented character classes, but the context always makes it clear which meaning is intended. For example `^aeiou` matches any complete line that does not contain a lowercase vowel.

- How do you tell which meaning of `.` is intended?
- Can you always replace a regular expression using the `^` and `$` operators by an equivalent expression that does not use either of these operators?

Exercise 3.3.11 The UNIX shell command `sh` uses the operators in Fig. 3.9 in filename expressions to describe sets of file names. For example the filename expression `*o` matches all file names ending in `o`, `sort1` matches all file names of the form `sort1 c` where c is any character. Show how `sh` filename

EXPRESSION	MATCHES	EXAMPLE
' <i>s</i> '	string <i>s</i> literally	
\ <i>c</i>	character <i>c</i> literally	
*	any string	o
.	any character	sort1
<i>s</i>	any character in <i>s</i>	sort1 cso

Figure 3.9 Filename expressions used by the shell command `sh`

expressions can be replaced by equivalent regular expressions using only the basic union, concatenation, and closure operators.

Exercise 3.3.12 SQL allows a rudimentary form of patterns in which two characters have special meaning: underscore `_` stands for any one character and percent sign `%` stands for any string of 0 or more characters. In addition, the programmer may define any character, say *e*, to be the escape character, so an *e* preceding `_` or another *e* gives the character that follows its literal meaning. Show how to express any SQL pattern as a regular expression, given that we know which character is the escape character.

3.4 Recognition of Tokens

In the previous section we learned how to express patterns using regular expressions. Now we must study how to take the patterns for all the needed tokens and build a piece of code that examines the input string and finds a prefix that is a lexeme matching one of the patterns. Our discussion will make use of the following running example:

<i>stmt</i>		if <i>expr</i> then <i>stmt</i>
		if <i>expr</i> then <i>stmt</i> else <i>stmt</i>
<i>expr</i>		<i>term</i> relop <i>term</i>
		<i>term</i>
<i>term</i>		id
		number

Figure 3.10 A grammar for branching statements

Example 3.8 The grammar fragment of Fig. 3.10 describes a simple form of branching statements and conditional expressions. This syntax is similar to that of the language Pascal, in that **then** appears explicitly after conditions.

3.4 RECOGNITION OF TOKENS

For **relop** we use the comparison operators of languages like Pascal or SQL where `=` is equals and `<>` is not equals because it presents an interesting structure of lexemes

The terminals of the grammar which are **if then else relop id** and **number** are the names of tokens as far as the lexical analyzer is concerned. The patterns for these tokens are described using regular definitions as in Fig. 3.11. The patterns for *id* and *number* are similar to what we saw in Example 3.7

<i>digit</i>	0 9
<i>digits</i>	<i>digit</i>
<i>number</i>	<i>digits</i> <i>digits</i> E <i>digits</i>
<i>letter</i>	A Za z
<i>id</i>	<i>letter letter</i> <i>digit</i>
<i>if</i>	if
<i>then</i>	then
<i>else</i>	else
<i>relop</i>	

Figure 3.11 Patterns for tokens of Example 3.8

For this language the lexical analyzer will recognize the keywords **if then** and **else** as well as lexemes that match the patterns for *relop id* and *number*. To simplify matters we make the common assumption that keywords are also *reserved words* that is they are not identifiers even though their lexemes match the pattern for identifiers.

In addition we assign the lexical analyzer the job of stripping out white space by recognizing the token *ws* defined by

$$ws = \text{blank} \mid \text{tab} \mid \text{newline}$$

Here **blank tab** and **newline** are abstract symbols that we use to express the ASCII characters of the same names. Token *ws* is different from the other tokens in that when we recognize it we do not return it to the parser but rather restart the lexical analysis from the character that follows the whitespace. It is the following token that gets returned to the parser.

Our goal for the lexical analyzer is summarized in Fig. 3.12. That table shows for each lexeme or family of lexemes which token name is returned to the parser and what attribute value as discussed in Section 3.1.3 is returned. Note that for the six relational operators symbolic constants LT LE and so on are used as the attribute value in order to indicate which instance of the token **relop** we have found. The particular operator found will influence the code that is output from the compiler. □

LEXEMES	TOKEN NAME	ATTRIBUTE VALUE
Any <i>ws</i>		
if	if	
then	then	
else	else	
Any <i>id</i>	id	Pointer to table entry
Any <i>number</i>	number	Pointer to table entry
	relop	LT
	relop	LE
	relop	EQ
	relop	NE
	relop	GT
	relop	GE

Figure 3.12 Tokens, their patterns, and attribute values

3.4.1 Transition Diagrams

As an intermediate step in the construction of a lexical analyzer, we first convert patterns into stylized flowcharts called transition diagrams. In this section we perform the conversion from regular expression patterns to transition diagrams by hand, but in Section 3.6 we shall see that there is a mechanical way to construct these diagrams from collections of regular expressions.

Transition diagrams have a collection of nodes or circles called *states*. Each state represents a condition that could occur during the process of scanning the input looking for a lexeme that matches one of several patterns. We may think of a state as summarizing all we need to know about what characters we have seen between the *lexemeBegin* pointer and the *forward* pointer, as in the situation of Fig. 3.3.

Edges are directed from one state of the transition diagram to another. Each edge is *labeled* by a symbol or set of symbols. If we are in some state *s* and the next input symbol is *a*, we look for an edge out of state *s* labeled by *a* and perhaps by other symbols as well. If we find such an edge, we advance the *forward* pointer and enter the state of the transition diagram to which that edge leads. We shall assume that all our transition diagrams are *deterministic*, meaning that there is never more than one edge out of a given state with a given symbol among its labels. Starting in Section 3.5 we shall relax the condition of determinism, making life much easier for the designer of a lexical analyzer, although trickier for the implementer. Some important conventions about transition diagrams are

1. Certain states are said to be *accepting* or *final*. These states indicate that a lexeme has been found, although the actual lexeme may not consist of all positions between the *lexemeBegin* and *forward* pointers. We always

3.4 RECOGNITION OF TOKENS

indicate an accepting state by a double circle and if there is an action to be taken – typically returning a token and an attribute value to the parser – we shall attach that action to the accepting state

- 2 In addition, if it is necessary to retract the *forward* pointer one position – i.e. the lexeme does not include the symbol that got us to the accepting state – then we shall additionally place a `<` near that accepting state. In our example, it is never necessary to retract *forward* by more than one position, but if it were, we could attach any number of `<`s to the accepting state.
- 3 One state is designated the *start state* or *initial state*; it is indicated by an edge labeled `start` entering from nowhere. The transition diagram always begins in the start state before any input symbols have been read.

Example 3.9 Figure 3.13 is a transition diagram that recognizes the lexemes matching the token **relop**. We begin in state 0 – the start state. If we see `<` as the first input symbol, then among the lexemes that match the pattern for **relop** we can only be looking at `<` or `<=`. We therefore go to state 1 and look at the next character. If it is `=`, then we recognize lexeme `<=`, enter state 2, and return the token **relop** with attribute LE – the symbolic constant representing this particular comparison operator. If in state 1 the next character is `>`, then instead we have lexeme `<>` and enter state 3 to return an indication that the not equals operator has been found. On any other character, the lexeme is `<` and we enter state 4 to return that information. Note, however, that state 4 has a `<` to indicate that we must retract the input one position.

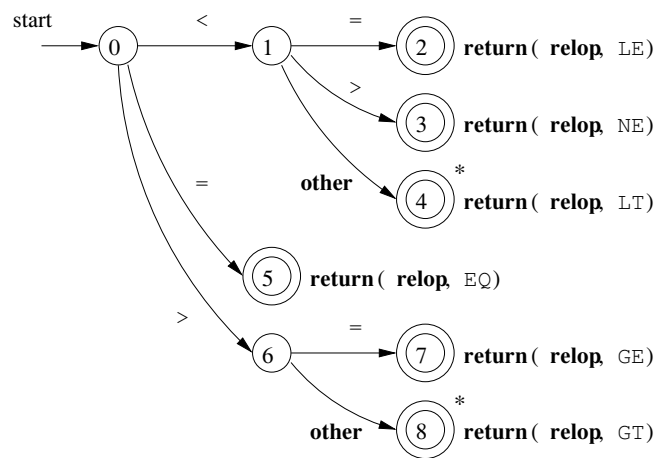


Figure 3.13 Transition diagram for **relop**

On the other hand, if in state 0 the first character we see is `=`, then this one character must be the lexeme `=`. We immediately return that fact from state 5.

The remaining possibility is that the first character is `if`. Then we must enter state 6 and decide on the basis of the next character whether the lexeme is `if` if we next see the `if` sign or just `if` on any other character. Note that if in state 0 we see any character besides `if` or `if` we can not possibly be seeing a `relop` lexeme so this transition diagram will not be used. $\square$

3.4.2 Recognition of Reserved Words and Identifiers

Recognizing keywords and identifiers presents a problem. Usually keywords like `if` or `then` are reserved as they are in our running example so they are not identifiers even though they *look* like identifiers. Thus although we typically use a transition diagram like that of Fig. 3.14 to search for identifier lexemes this diagram will also recognize the keywords `if`, `then`, and `else` of our running example.

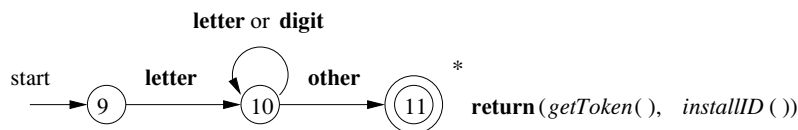


Figure 3.14 A transition diagram for `id`s and keywords

There are two ways that we can handle reserved words that look like identifiers.

1. Install the reserved words in the symbol table initially. A field of the symbol table entry indicates that these strings are never ordinary identifiers and tells which token they represent. We have supposed that this method is in use in Fig. 3.14. When we find an identifier, a call to *installID* places it in the symbol table if it is not already there and returns a pointer to the symbol table entry for the lexeme found. Of course any identifier not in the symbol table during lexical analysis cannot be a reserved word so its token is `id`. The function *getToken* examines the symbol table entry for the lexeme found and returns whatever token name the symbol table says this lexeme represents—either `id` or one of the keyword tokens that was initially installed in the table.
2. Create separate transition diagrams for each keyword. An example for the keyword `then` is shown in Fig. 3.15. Note that such a transition diagram consists of states representing the situation after each successive letter of the keyword is seen, followed by a test for a nonletter or digit, i.e., any character that cannot be the continuation of an identifier. It is necessary to check that the identifier has ended, or else we would return token `then` in situations where the correct token was `id` with a lexeme like `thenextvalue` that has `then` as a proper prefix. If we adopt this approach, then we must prioritize the tokens so that the reserved word

3.4 RECOGNITION OF TOKENS

tokens are recognized in preference to **id** when the lexeme matches both patterns. We *do not* use this approach in our example, which is why the states in Fig. 3.15 are unnumbered.

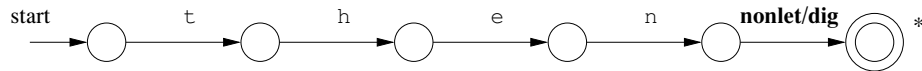


Figure 3.15 Hypothetical transition diagram for the keyword **then**

3.4.3 Completion of the Running Example

The transition diagram for **ids** that we saw in Fig. 3.14 has a simple structure. Starting in state 9, it checks that the lexeme begins with a letter and goes to state 10 if so. We stay in state 10 as long as the input contains letters and digits. When we first encounter anything but a letter or digit, we go to state 11 and accept the lexeme found. Since the last character is not part of the identifier, we must retract the input one position, and as discussed in Section 3.4.2, we enter what we have found in the symbol table and determine whether we have a keyword or a true identifier.

The transition diagram for token **number** is shown in Fig. 3.16, and is so far the most complex diagram we have seen. Beginning in state 12, if we see a digit, we go to state 13. In that state, we can read any number of additional digits. However, if we see anything but a digit, dot, or E, we have seen a number in the form of an integer. 123 is an example. That case is handled by entering state 20, where we return token **number** and a pointer to a table of constants where the found lexeme is entered. These mechanics are not shown on the diagram but are analogous to the way we handled identifiers.

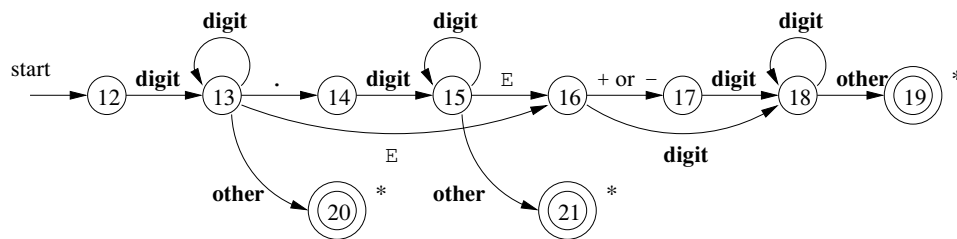


Figure 3.16 A transition diagram for unsigned numbers

If we instead see a dot in state 13, then we have an optional fraction. State 14 is entered, and we look for one or more additional digits; state 15 is used for that purpose. If we see an E, then we have an optional exponent whose recognition is the job of states 16 through 19. Should we, in state 15, instead see anything but E or a digit, then we have come to the end of the fraction; there is no exponent, and we return the lexeme found via state 21.

The final transition diagram shown in Fig. 3.17 is for whitespace. In that diagram, we look for one or more whitespace characters represented by **delim** in that diagram—typically these characters would be blank, tab, newline, and perhaps other characters that are not considered by the language design to be part of any token.

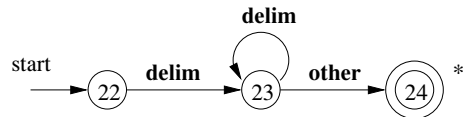


Figure 3.17 A transition diagram for whitespace

Note that in state 24, we have found a block of consecutive whitespace characters followed by a nonwhitespace character. We retract the input to begin at the nonwhitespace, but we do not return to the parser. Rather, we must restart the process of lexical analysis after the whitespace.

3.4.4 Architecture of a Transition Diagram Based Lexical Analyzer

There are several ways that a collection of transition diagrams can be used to build a lexical analyzer. Regardless of the overall strategy, each state is represented by a piece of code. We may imagine a variable **state** holding the number of the current state for a transition diagram. A switch based on the value of **state** takes us to code for each of the possible states, where we find the action of that state. Often, the code for a state is itself a switch statement or multiway branch that determines the next state by reading and examining the next input character.

Example 3.10 In Fig. 3.18 we see a sketch of **getRelop**, a C function whose job is to simulate the transition diagram of Fig. 3.13 and return an object of type **TOKEN** that is a pair consisting of the token name, which must be **relop** in this case, and an attribute value, the code for one of the six comparison operators in this case. **getRelop** first creates a new object **retToken** and initializes its first component to **RELOP**, the symbolic code for token **relop**.

We see the typical behavior of a state in case 0, the case where the current state is 0. A function **nextChar** obtains the next character from the input and assigns it to local variable *c*. We then check *c* for the three characters we expect to find, making the state transition dictated by the transition diagram of Fig. 3.13 in each case. For example, if the next input character is `<`, we go to state 5.

If the next input character is not one that can begin a comparison operator, then a function **fail** is called. What **fail** does depends on the global error recovery strategy of the lexical analyzer. It should reset the **forward** pointer to **lexemeBegin** in order to allow another transition diagram to be applied to

3.4 RECOGNITION OF TOKENS

```

TOKEN getRelop

    TOKEN retToken  new RELOP
    while 1         repeat character processing until a return
                      or failure occurs
        switch state
            case 0 c  nextChar
                if c      state 1
                else if c  state 5
                else if c  state 6
                else fail   lexeme is not a relop
                break
            case 1
            case 8 retract
                retToken attribute  GT
                return retToken

```

Figure 3.18 Sketch of implementation of **relop** transition diagram

the true beginning of the unprocessed input. It might then change the value of **state** to be the start state for another transition diagram which will search for another token. Alternatively, if there is no other transition diagram that remains unused, **fail** could initiate an error correction phase that will try to repair the input and find a lexeme as discussed in Section 3.1.4.

We also show the action for state 8 in Fig. 3.18. Because state 8 bears a we must retract the input pointer one position, i.e., put *c* back on the input stream. That task is accomplished by the function **retract**. Since state 8 represents the recognition of lexeme, we set the second component of the returned object, which we suppose is named **attribute**, to **GT**, the code for this operator. □

To place the simulation of one transition diagram in perspective, let us consider the ways code like Fig. 3.18 could fit into the entire lexical analyzer.

1. We could arrange for the transition diagrams for each token to be tried sequentially. Then the function **fail** of Example 3.10 resets the pointer **forward** and starts the next transition diagram each time it is called. This method allows us to use transition diagrams for the individual keywords like the one suggested in Fig. 3.15. We have only to use these before we use the diagram for **id** in order for the keywords to be reserved words.

- 2 We could run the various transition diagrams in parallel feeding the next input character to all of them and allowing each one to make whatever transitions it required. If we use this strategy we must be careful to resolve the case where one diagram finds a lexeme that matches its pattern while one or more other diagrams are still able to process input. The normal strategy is to take the longest prefix of the input that matches any pattern. That rule allows us to prefer identifier `thenext` to keyword `then` or the operator `to` for example.
- 3 The preferred approach and the one we shall take up in the following sections is to combine all the transition diagrams into one. We allow the transition diagram to read input until there is no possible next state and then take the longest lexeme that matched any pattern as we discussed in item 2 above. In our running example this combination is easy because no two tokens can start with the same character i.e. the first character immediately tells us which token we are looking for. Thus we could simply combine states 0, 9, 12 and 22 into one start state leaving other transitions intact. However in general the problem of combining transition diagrams for several tokens is more complex as we shall see shortly.

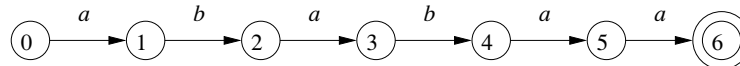
3.4.5 Exercises for Section 3.4

Exercise 3.4.1 Provide transition diagrams to recognize the same languages as each of the regular expressions in Exercise 3.3.2.

Exercise 3.4.2 Provide transition diagrams to recognize the same languages as each of the regular expressions in Exercise 3.3.5.

The following exercises up to Exercise 3.4.12 introduce the Aho Corasick algorithm for recognizing a collection of keywords in a text string in time proportional to the length of the text and the sum of the length of the keywords. This algorithm uses a special form of transition diagram called a *trie*. A trie is a tree structured transition diagram with distinct labels on the edges leading from a node to its children. Leaves of the trie represent recognized keywords.

Knuth, Morris, and Pratt presented an algorithm for recognizing a single keyword $b_1b_2 \dots b_n$ in a text string. Here the trie is a transition diagram with $n + 1$ states, 0 through n . State 0 is the initial state and state n represents acceptance (that is, discovery of the keyword). From each state s from 0 through $n - 1$ there is a transition to state $s + 1$ labeled by symbol b_{s+1} . For example the trie for the keyword `ababaa` is



In order to process text strings rapidly and search those strings for a keyword, it is useful to define for keyword $b_1b_2 \dots b_n$ and position s in that keyword corresponding to state s of its trie, a *failure function* $f[s]$ computed as in

3.4 RECOGNITION OF TOKENS

Fig. 3.19 The objective is that $b_1b_2 \dots b_{f(s)}$ is the longest proper pre x of $b_1b_2 \dots b_s$ that is also a su x of $b_1b_2 \dots b_s$. The reason $f(s)$ is important is that if we are trying to match a text string for $b_1b_2 \dots b_n$ and we have matched the first s positions but we then fail i.e. the next position of the text string does not hold b_{s+1} then $f(s)$ is the longest pre x of $b_1b_2 \dots b_n$ that could possibly match the text string up to the point we are at. Of course the next character of the text string must be $b_{f(s)+1}$ or else we still have problems and must consider a yet shorter pre x which will be $b_{f(f(s))}$.

```

1  t ← 0
2  f[1] ← 0
3  for s ← 1 to n do
4      while t > 0 and b[s+1] ≠ b[t+1] do
5          if b[s+1] = b[t+1] then
6              t ← t + 1
7              f[s] ← t
8          else f[s] ← 0
9      }
10 }
```

Figure 3.19 Algorithm to compute the failure function for keyword $b_1b_2 \dots b_n$

As an example the failure function for the trie constructed above for **ababaa** is

s	1	2	3	4	5	6
f[s]	0	0	1	2	3	1

For instance states 3 and 1 represent pre xes **aba** and **a** respectively $f(3) = 1$ because **a** is the longest proper pre x of **aba** that is also a su x of **aba**. Also $f(2) = 0$ because the longest proper pre x of **ab** that is also a su x is the empty string.

Exercise 3.4.3 Construct the failure function for the strings

- a abababaab
- b aaaaaa
- c abbaabb

Exercise 3.4.4 Prove by induction on s that the algorithm of Fig. 3.19 correctly computes the failure function.

Exercise 3.4.5 Show that the assignment $t ← f(t)$ in line 4 of Fig. 3.19 is executed at most n times. Show that therefore the entire algorithm takes only $O(n)$ time on a keyword of length n .

CHAPTER 3 LEXICAL ANALYSIS

Having computed the failure function for a keyword $b_1b_2 \dots b_n$ we can scan a string $a_1a_2 \dots a_m$ in time $O(m)$ to tell whether the keyword occurs in the string. The algorithm shown in Fig. 3.20 slides the keyword along the string trying to make progress by matching the next character of the keyword with the next character of the string. If it cannot do so after matching s characters, then it slides the keyword right $s - f(s)$ positions so only the first $f(s)$ characters of the keyword are considered matched with the string.

```

1  s ← 0
2  for i ← 1 to m do {
3      while s > 0 and  $a_i \neq b_s$  do s ← f(s)
4      if  $a_i = b_s$  then s ← s + 1
5      if s = n then return yes
6  }
7  return no

```

Figure 3.20 The KMP algorithm tests whether string $a_1a_2 \dots a_m$ contains a single keyword $b_1b_2 \dots b_n$ as a substring in $O(m + n)$ time.

Exercise 3.4.6 Apply Algorithm KMP to test whether keyword **ababaa** is a substring of

a **abababaab**

b **abababbbaa**

Exercise 3.4.7 Show that the algorithm of Fig. 3.20 correctly tells whether the keyword is a substring of the given string. *Hint* proceed by induction on i . Show that for all i the value of s after line 4 is the length of the longest prefix of the keyword that is a suffix of $a_1a_2 \dots a_i$.

Exercise 3.4.8 Show that the algorithm of Fig. 3.20 runs in time $O(m + n)$ assuming that function f is already computed and its values stored in an array indexed by s .

Exercise 3.4.9 The *Fibonacci strings* are defined as follows

- 1 $s_1 = \text{b}$
- 2 $s_2 = \text{a}$
- 3 $s_k = s_{k-1}s_{k-2}$ for $k \geq 2$

For example $s_3 = \text{ab}$, $s_4 = \text{aba}$ and $s_5 = \text{abaab}$.

- a What is the length of s_n ?

3.4 RECOGNITION OF TOKENS

- Construct the failure function for s_6
- Construct the failure function for s_7
- Show that the failure function for any s_n can be expressed by $f[1]$
 $f[2] = 0$ and for $2 \leq j \leq |s_n|$ $f[j]$ is $j - |s_{k-1}|$ where k is the largest integer such that $|s_k| = j - 1$
- In the KMP algorithm what is the largest number of consecutive applications of the failure function when we try to determine whether keyword s_k appears in text string s_{k-1}

Aho and Corasick generalized the KMP algorithm to recognize any of a set of keywords in a text string. In this case the trie is a true tree with branching from the root. There is one state for every string that is a prefix not necessarily proper of any keyword. The parent of a state corresponding to string $b_1b_2 \dots b_k$ is the state that corresponds to $b_1b_2 \dots b_{k-1}$. A state is accepting if it corresponds to a complete keyword. For example Fig. 3.21 shows the trie for the keywords **he she his** and **hers**.

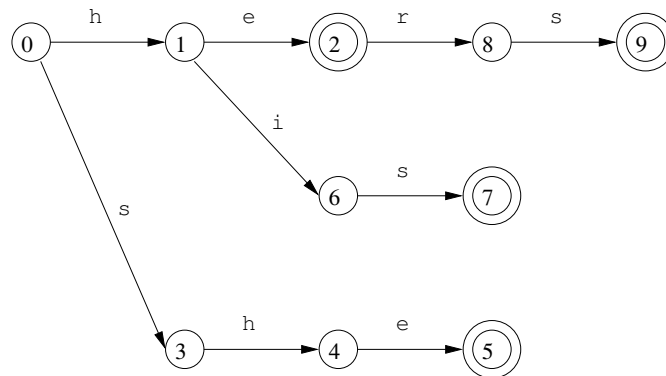


Figure 3.21 Trie for keywords **he she his hers**

The failure function for the general trie is defined as follows. Suppose s is the state that corresponds to string $b_1b_2 \dots b_n$. Then $f[s]$ is the state that corresponds to the longest proper suffix of $b_1b_2 \dots b_n$ that is also a prefix of some keyword. For example the failure function for the trie of Fig. 3.21 is

s	1	2	3	4	5	6	7	8	9
$f[s]$	0	0	0	1	2	0	3	0	3

Exercise 3.4.10 Modify the algorithm of Fig. 3.19 to compute the failure function for general tries. *Hint* The major difference is that we cannot simply test for equality or inequality of b_{s-1} and b_{t-1} in lines 4 and 5 of Fig. 3.19. Rather from any state there may be several transitions out on several characters as there are transitions on both **e** and **i** from state 1 in Fig. 3.21. Any of

those transitions could lead to a state that represents the longest suffix that is also a prefix

Exercise 3.4.11 Construct the tries and compute the failure function for the following sets of keywords

- a `aaa abaaa and ababaaa`
- b `all fall fatal llama and lame`
- c `pipe pet item temper and perpetual`

Exercise 3.4.12 Show that your algorithm from Exercise 3.4.10 still runs in time that is linear in the sum of the lengths of the keywords

3.5 The Lexical Analyzer Generator Lex

In this section we introduce a tool called **Lex** or in a more recent implementation **Flex** that allows one to specify a lexical analyzer by specifying regular expressions to describe patterns for tokens. The input notation for the **Lex** tool is referred to as the *Lex language* and the tool itself is the *Lex compiler*. Behind the scenes the Lex compiler transforms the input patterns into a transition diagram and generates code in a file called `lex.yy.c` that simulates this transition diagram. The mechanics of how this translation from regular expressions to transition diagrams occurs is the subject of the next sections; here we only learn the Lex language.

3.5.1 Use of Lex

Figure 3.22 suggests how **Lex** is used. An input file which we call `lex.l` is written in the Lex language and describes the lexical analyzer to be generated. The Lex compiler transforms `lex.l` to a C program in a file that is always named `lex.yy.c`. The latter file is compiled by the C compiler into a file called `a.out` as always. The C compiler output is a working lexical analyzer that can take a stream of input characters and produce a stream of tokens.

The normal use of the compiled C program referred to as `a.out` in Fig. 3.22 is as a subroutine of the parser. It is a C function that returns an integer which is a code for one of the possible token names. The attribute value, whether it be another numeric code, a pointer to the symbol table, or nothing, is placed in a global variable `yylval`² which is shared between the lexical analyzer and parser, thereby making it simple to return both the name and an attribute value of a token.

²Incidentally, the `yy` that appears in `yylval` and `lex.yy.c` refers to the **Yacc** parser generator which we shall describe in Section 4.9 and which is commonly used in conjunction with **Lex**.